

Projet XYZ

Table des matières

1	Analyse préliminaire	4
1.1	Introduction	4
1.2	Objectifs.....	5
1.2.1	Livrable	5
1.2.2	Objectif du produit.....	5
1.3	Gestion de projet	6
1.4	Planification initiale	7
2	Analyse / Conception.....	10
2.1	Contexte produit	10
2.1.1	Utilisation	11
2.2	Contexte technique.....	11
2.2.1	Opérationnel	11
2.2.2	Environnement de développement	13
2.2.3	Environnement de production	14
2.2.4	Justification des choix.....	14
2.3	Dossier de conception	15
2.3.1	Zonings.....	15
2.3.2	Maquettes.....	18
2.3.3	Modèle de base de données	22
2.3.4	Analyse fonctionnelle.....	26
2.4	Stratégie de test.....	29
2.5	Risques techniques	30
2.6	Planification	Erreur ! Signet non défini.
3	Réalisation.....	31
3.1	Point de design spécifique.....	31
3.1.1	Gestion de l'authentification.....	31
3.1.2	Gestion des routes frontend	37
3.2	Dossier de réalisation	53
3.3	Description des tests effectués	56

3.4	Erreurs restantes	60
3.5	Liste des documents fournis	60
4	Conclusions	61
5	Annexes.....	65
5.1	Résumé du rapport du TPI / version succincte de la documentation	65
5.2	Sources – Bibliographie.....	65
5.3	Journal de travail	65
5.4	Manuel d'Installation	65
5.5	Manuel d'Utilisation.....	66
5.6	Archives du projet.....	66

1 Analyse préliminaire

1.1 Introduction

L'objectif principal de ce projet est de créer une application de simulation de saison de Formule 1. Elle se présentera sous forme de jeu afin de divertir ses utilisateurs. Cette application sera développée sous forme d'application web en utilisant le langage de programmation Javascript.

Ce projet se fait dans le cadre du TPI. Cela veut dire que le projet sera réalisé en suivant les modalités du TPI. Le dossier de projet ainsi que la présentation faisant suite à ce projet seront aussi évalués en suivant la grille d'évaluation TPI VD.

Plusieurs projets ont été réalisés en prévision de ce TPI. Le premier, un projet de monitoring de ruche d'abeille, n'avait aucun lien avec la technologie qui sera utilisée ni aucun rapport avec le thème (La formule 1) de ce TPI. Cependant, ce projet, ayant été évalué avec la grille d'évaluation du TPI, m'a permis de me familiariser avec cette dite grille ainsi que ce que devrait contenir le dossier de projet. Le second projet la création d'un site web de shopping avec un back-office était, lui, un projet non-officiel. Cela veut dire que le projet ne suivait pas de cahier des charges ni d'évaluation finale mais me servait uniquement de moyen d'assimiler la technologie qui sera utilisée dans ce TPI et d'y gagner en aisance. Plusieurs fonctionnalité ou bout de code seront repris de ce projet.

1.2 Objectifs

1.2.1 Livrable

Une planification initiale contenant un diagramme de Gantt devra être livrée aux 2 experts ainsi qu'au chef de projet par courriel électronique sous le format PDF à la fin du premier jour (2 mai 2025).

Le rapport de projet en l'état devra être livré 3 fois par semaine le lundi, le mercredi et le vendredi en fin de journée. Le rapport de projet sera livré sous forme de PDF dans le repo Git dans un dossier dédié à la documentation.

Le livrable finale devra contenir :

- Le journal de travail avec mention de tout ce qui a été accompli durant le projet devra aussi être livrée.
- Le code source de l'application devra être livré aux 2 experts ainsi qu'au chef de projet. La livraison se fera sous la forme de partage de lien au dépôt Git.
- Les scripts qui serviront la création de base de données ainsi que des données de test
- Un guide de déploiement de l'application sur l'infrastructure locale de l'ETML

A la fin du délai imparti pour la réalisation du TPI, c'est à dire avant le 2 juin 2025 à 11h20, une version électronique du dossier de projet devra être transmise aux 2 experts ainsi qu'au chef de projet. En parallèle, une copie papier du rapport devra être fournie sans délai en trois exemplaires. Elle devra être en tout point identique à la version électronique.

1.2.2 Objectif du produit.

Comme énoncé dans l'introduction, l'objectif du projet est de créer une application de simulation de saison de Formule 1. Etant donné que le descriptif peut porter à confusion, je vais décrire, ici, ce à quoi consiste cette application. En effet, il ne s'agit pas de simuler la saison de Formule 1 actuelle, mais de donner la possibilité à un

utilisateur de se divertir en choisissant une écurie, choisir les 2 pilotes qui composeront cette écurie et de simuler une saison entière. Une fois la saison lancée, l'utilisateur n'aura aucune influence sur le résultat des courses. Les résultats des courses seront aléatoires en prenant en compte le rating¹ du pilote. Les courses comprendront au moins un accident et un abandon. Cela sera un jeu en solo. Cela veut dire qu'aucun autre joueur pourra prendre part à la session de jeu de l'utilisateur.

1.3 Gestion de projet

Une planification initiale sera faite le premier jour de ce projet, directement après la réception du cahier des charges (la planification initiale est trouvable sous le chapitre 1.4).

La méthodologie utilisée lors de ce projet de TPI est celle dite *en cascade*. C'est-à-dire que lors de la planification initiale, le projet a été analysé pour en ressortir toutes les tâches qui seront réalisées. Les tâches ont été mises ensuite dans un ordre logique pour suivre la logique du *Waterfall* qui est qu'une tâche ne peut pas être réalisée avant que la précédente ne soit finie. En effet, il sera difficile de s'occuper de la gestion de course avant de s'occuper de la gestion des choix des pilotes et des écuries.

L'entièreté du projet sera disponible sur une repo github (Code source de l'application, planification, journal de travail, rapport, cahier des charges, schémas).

Les personnes impliquées dans ce projet sont :

- Gaël Sonney, chef de projet.
- Yves Bertino, expert n° 1
- Nemanja Pantic, expert n° 2

¹ Niveau de compétence du pilote

1.4 Planification initiale

Une planification a donc été faite le premier jour de ce TPI. Les tâches ont d'abord été séparées en plusieurs catégories :

- Entretien, les différents entretiens avec les experts ou le chef de projet.
- Analyse, analyse du projet ou d'une fonctionnalité.
- Réalisation, développement d'une fonctionnalité ou réalisation d'une tâche précise.
- Test, tester une fonctionnalité.
- Documentation, rédiger la partie de la documentation du projet.

Un découpage des tâches plus précises a ensuite été fait. Puis, pour chaque séquence de travail, une estimation des tâches qui seront réalisées ainsi que le temps passé dessus a été faite. Sachant qu'une séquence est définie par une demi-journée de travail

Comparaison entre les tâches planifiées et réalisées

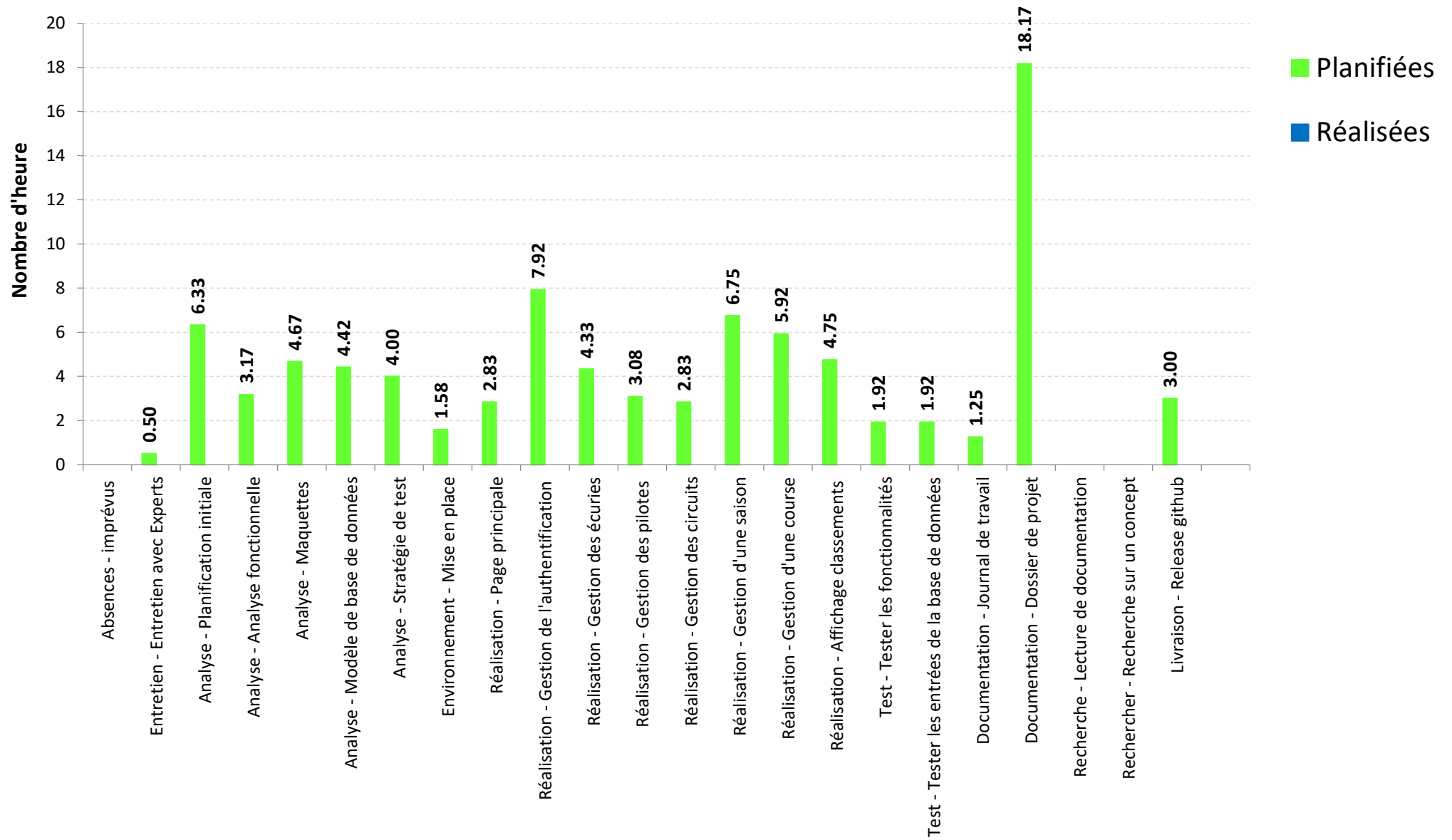


Figure 1: Planification initiale

A Garder à l'esprit que cette planification initiale n'est qu'une première idée de comment pourrait se dérouler le projet et qu'elle n'est là que pour donner une idée de l'ordre des tâches à effectuer ainsi que le temps estimé que prendra chacune d'elles. Il est très peu probable que cette planification initiale soit respectée parfaitement. Des imprévus peuvent survenir, de nouvelles idées, des idées de réalisation différentes, ou une mauvaise estimation du temps peuvent en être la cause.

La période de réalisation de ce projet du TPI est du **vendredi 2 mai 2025 à 8h au lundi 2 juin 2025 à 11h25**.

Les horaires de travail sont :

Lundi	08h00 – 11h25	12h20 – 14h45
Mardi	-	-
Mercredi	08h00 – 12h15	13h10 – 16h35
Jeudi	-	13h10 – 16h35
Vendredi	08h00 – 12h15	13h10 – 16h35

Avec comme jours fériés :

- Lundi 26 mai après-midi, Examen eCG
- Jeudi 29 mai, Ascension
- Vendredi 30 mai, Pont de l'Ascension

2 Analyse / Conception

2.1 Contexte produit

Le Produit sera un accès à un site internet. L'utilisateur devra en premier lieu se connecter afin de pouvoir profiter des fonctionnalités. Le site web sera principalement développé pour les navigateurs web via ordinateur. Si un maximum sera fait pour rendre le site web plus modulable, il ne sera pas pensé pour avoir un responsive design complet. Le site sera tout de même accessible sur un navigateur web depuis un téléphone mobile, mais la vue ne sera pas totalement adaptée.

L'application aura 2 types d'utilisateur :

- Les utilisateurs simples, qui pourront jouer et consulter les données
- Les administrateurs, qui pourront modifier les données.

Actions / Rôles	User	Admin
Voir les informations des écuries	✓	✓
Modifier les informations des écuries	✗	✓
Supprimer une écurie	✗	✓
Ajouter une écurie	✗	✓
Voir les informations des pilotes	✓	✓
Modifier les information des pilotes	✗	✓
Supprimer un pilote	✗	✓
Ajouter un pilote	✗	✓
Voir les informations des circuits	✓	✓
Modifier les informations des circuits	✗	✓

Supprimer un circuit	×	✓
Ajouter un circuit	×	✓
Lancer une saison	✓	✓

2.1.1 Utilisation

Prenons l'exemple d'utilisation d'un utilisateur simple. Il devra en premier lieu se connecter au site internet avec ses identifiants. S'il n'en a pas, il aura la possibilité de créer un compte. Ensuite, il aura l'occasion de consulter les données des pilotes, des circuits ou des écuries s'il en a l'envie. Pour commencer une partie, il devra se rendre sur la page principale puis cliquer sur le bouton *Lancer une saison*. Le site web affichera ensuite une liste des écuries, il en choisira une. Puis lui sera affiché la liste des pilotes, il fera de même mais en choisira 2. Ce seront ses 2 pilotes qui courront pour lui dans l'écurie qu'il a choisie. Une fois confirmé, la saison débute. Chaque course se lanceront une à une, un classement des pilotes et des constructeurs sera affiché à la fin de chaque course. L'utilisateur devra, néanmoins, confirmer à chaque fin de course avant de passer à la suivante. A la fin de la saison, un classement final sera affiché. Le joueur se retrouvera donc comme au début de sa session de jeu, sur la page principale avec la possibilité de recommencer une partie.

2.2 Contexte technique

2.2.1 Opérationnel

Le produit final est divisé en plusieurs parties :

2.2.1.1 Frontend

La partie frontend², c'est-à-dire la partie qui sera visible par l'utilisateur, sera développée en utilisant Vite, React ainsi que TypeScript³, plus précisément avec le format TSX. Le TSX nous permettra d'intégrer directement du TypeScript dans nos balise JSX⁴.

Vite JS est, par définition, un serveur de développement utilisé pour faire tourner localement des applications web. Cela nous permettra aussi de faire un build de notre application afin de le déployer en production.

De plus, pour le style de l'application web, je vais utiliser tailwindcss, un framework css qui permettra de gagner du temps lors de la mise en style de notre site.

2.2.1.2 Backend

La partie backend⁵, la partie dont l'utilisateur n'a pas directement accès, sera développée avec Express JS. Express JS est un framework backend pour Node.js, qui nous permet de créer des API⁶, de gérer des requêtes HTTP et faire du routage. Son fonctionnement peut se décrire de cette façon :

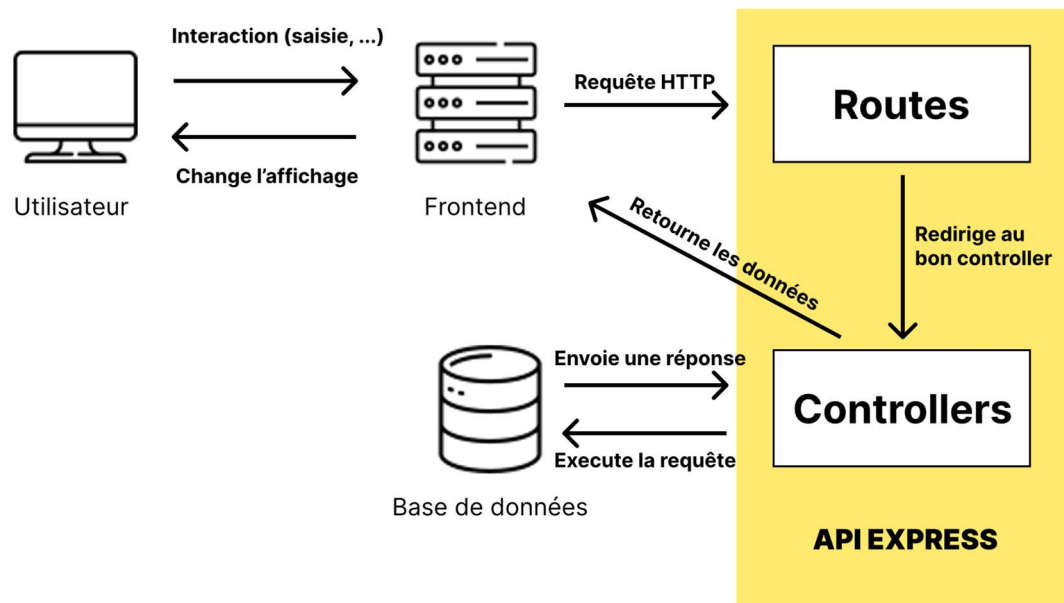
² Partie visible par l'utilisateur

³ Langage ressemblant au JS mais typé

⁴ Extension de syntaxe pour JS permettant d'y intégrer du HTML

⁵ Partie qui interagira avec la base de données

⁶ Application Programming Interface : permet la communication entre 2 applications



Prenons l'exemple où l'utilisateur se rends sur la page des pilotes. Lors du chargement de la page, la partie frontend effectue une requête HTTP sous forme de `http://localhost:3040/api/drivers` est effectuée (dans le cadre de l'environnement de développement). Dans le fichier `index.js` de notre backend toutes les routes API y sont définies. Parmi elles, la route `/api/drivers` est associé au fichier `routes/drivers.js`.

Ce fichier de routes importe les fonctions définies dans notre Controller et lie chaque routes avec une fonction.

Dans notre exemple, comme notre URL est simplement `api/drivers/` (aucun paramètre supplémentaire), c'est la fonction associée à la route « / » qui est exécutée. Dans notre cas c'est la fonction `getAllDrivers`. La fonction avec ce nom présent dans le `DriversController` va ensuite s'exécuter et va effectuer une requête SQL à notre base de données. Le *controller* va ensuite retourner les données reçues à la partie frontend qui va mettre à jour l'affichage et afficher la liste des pilotes

2.2.2 Environnement de développement

L'environnement de développement sera l'application lancée en local sur un PC. Le frontend sera déployé sur le port 5173 tandis que le backend le sera sur le 3040. Il faudra donc démarrer l'environnement via une console, celle de VSC par exemple, via

la commande « `npm run dev` » pour le frontend puis « `node index.js` » pour le backend. Cela veut dire que cet environnement n'est pas accessible depuis un poste extérieur. Comment son nom l'indique, c'est sur cet environnement que les fonctionnalités seront d'abord développées puis testées avant d'être remises sur l'environnement de production. J'ai principalement utilisé Google Chrome comme navigateur web pour développer mon application. Il se peut qu'il y ait des problèmes de style sur les autres navigateurs vu qu'au cours du projet, je n'ai pas eu le temps de vérifier et d'adapter l'application à tous les navigateurs web.

2.2.3 Environnement de production

L'environnement de production sera le serveur disponible à l'ETML. Ce serveur contient un service Nginx et Node ce qui nous permet d'héberger mon application web. Le port ouvert et accessible qui m'a été remis est le port 3040. L'application devra donc être modifiée pour que le backend et le frontend utilisent les 2 ce port attribué. L'application, étant hébergée sur le serveur de l'ETML, sera donc accessible par n'importe quel appareil connecté à Internet depuis un navigateur web. Le lien pour y accéder est *aki.w3.pm2etml.ch*. Lorsqu'une fonctionnalité a passé tous les tests sur l'environnement de dev, elle pourra être push sur le serveur. Un nouveau build de l'application devra être fait pour que les changements prennent effet.

2.2.4 Justification des choix

Pour ce projet, j'ai eu le choix d'utiliser soit React, soit PHP. C'est un choix auquel j'ai longuement réfléchi, majoritairement durant le pré-TPI pendant le développement de mon projet en React. L'avantage de PHP était que j'avais déjà pas mal de fonctionnalités prêtes, faites durant ma formation. J'étais aussi plus au courant des bonnes pratiques, les ayant apprises à l'ETML durant ma formation. Pour React et ViteJS, le danger était que j'ai appris tout ça en autodidacte, et donc, n'était pas forcément au courant des bonnes pratiques, mais faisais uniquement ce qui me semblait logique ou ce que je trouvais sur les différents forums.

J'ai finalement choisi d'utiliser React pour sa simplicité et le fait que j'allais faire plusieurs pages similaires et que le fonctionnement de React avec ses composants était une solution plus adéquates. Sa popularité fait que beaucoup de forums et de documentations sont trouvables sur internet.

J'ai aussi choisi React pour sa modernité. React est un des framework les plus utilisés. Le maîtriser me permet d'agrandir mes compétences et mes opportunités professionnelles futures. Ce projet en est donc une bonne opportunité est y apporte grandement.

2.3 Dossier de conception

Dans ce chapitre de concept, nous verrons tout ce qui est zonings, maquettes du site web, les différents schémas dont ceux de la base de données et ceux du fonctionnement des fonctionnalités du site web.

2.3.1 Zonings

Les zonings sont les premières esquisses graphiques des pages du sites web permettant de visualiser l'organisation des différents éléments qui seront présents sur les pages. Les zonings ont été réalisé sur Figma.

2.3.1.1 Page principale

Sur la page principale, on trouvera la barre de navigation en haut de notre écran qui contiendra les différents boutons pour ensuite naviguer à travers les différentes pages. Au centre de l'écran se trouvera la *zone de jeu*. C'est ici que se présentera le terrain de jeu pour notre utilisateur. Elle contiendra par exemple le déroulement de la course, ou bien le classement de la course une fois la course terminée.



Figure 2: Zoning page principale

2.3.1.2 Liste des pilotes

Cette page contiendra, comme toutes les autres, la barre de navigation horizontale en haut de l'écran. La liste des pilotes y figurera. Un simple utilisateur pourra ne pourra que consulter ces informations. Seul un utilisateur ayant comme rôle « Administrateur » pourra ajouter, modifier ou supprimer un des pilotes.

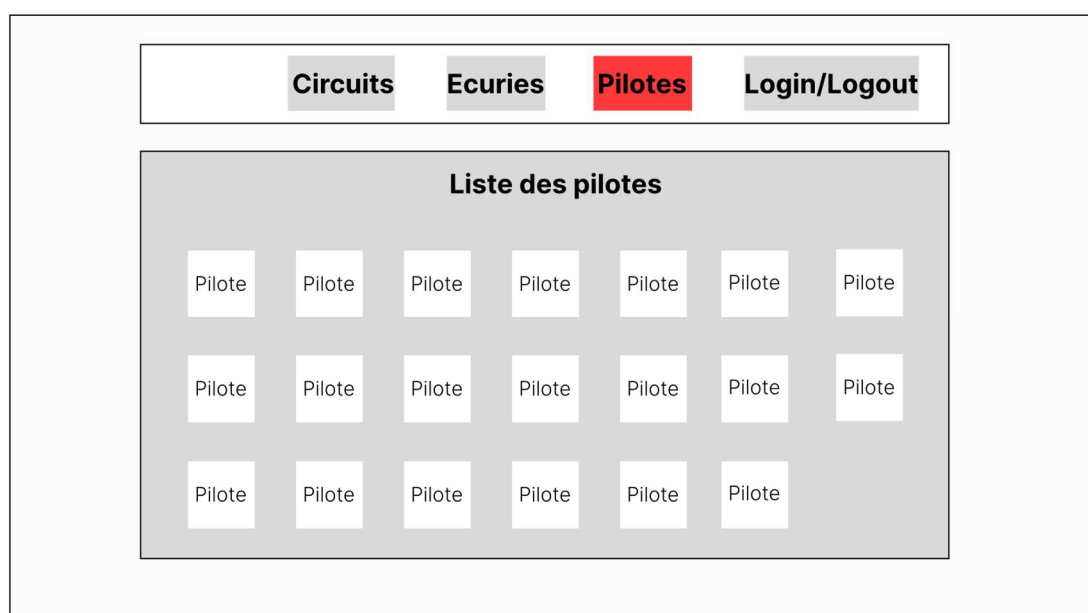


Figure 3: Zoning liste pilotes

2.3.1.3 Liste des écurie et des circuits

Ces pages sont relativement similaires à celle des pilotes dans leur fonctionnement. C'est-à-dire que seul un utilisateur de type « Administrateur » pourra y faire des modifications.

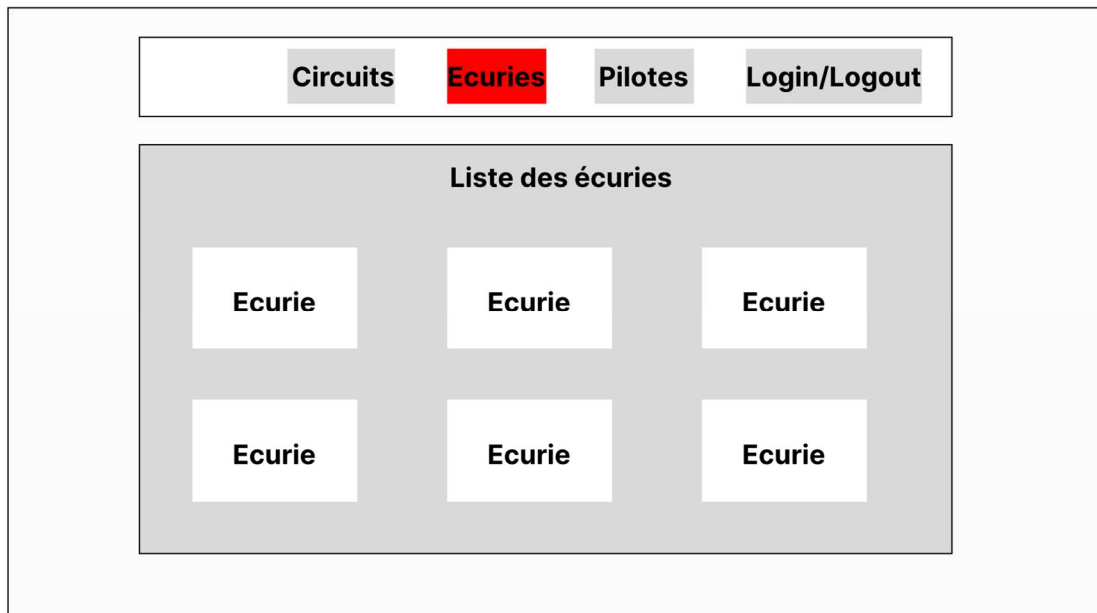


Figure 4: Zoning liste écuries

Etant donné que le zoning de la liste des circuits ressemble énormément à celui des écuries, j'ai trouvé plus pertinent de ne pas le mettre dans le rapport. Cependant il sera bien évidemment mis en annexe.

2.3.2 Maquettes

Les maquettes sont beaucoup plus complètes que les zonings et se rapproche plus du résultat final. Elles ont, comme pour les zonings, été faite sur Figma. Par soucis de pertinence, pas toutes les maquettes sont présentes dans ce chapitre étant donné que plusieurs pages se ressemblent beaucoup (page des pilotes / page des écuries). Vous pourrez trouver l'ensemble de toutes les maquettes en annexe.

2.3.2.1 Page principale

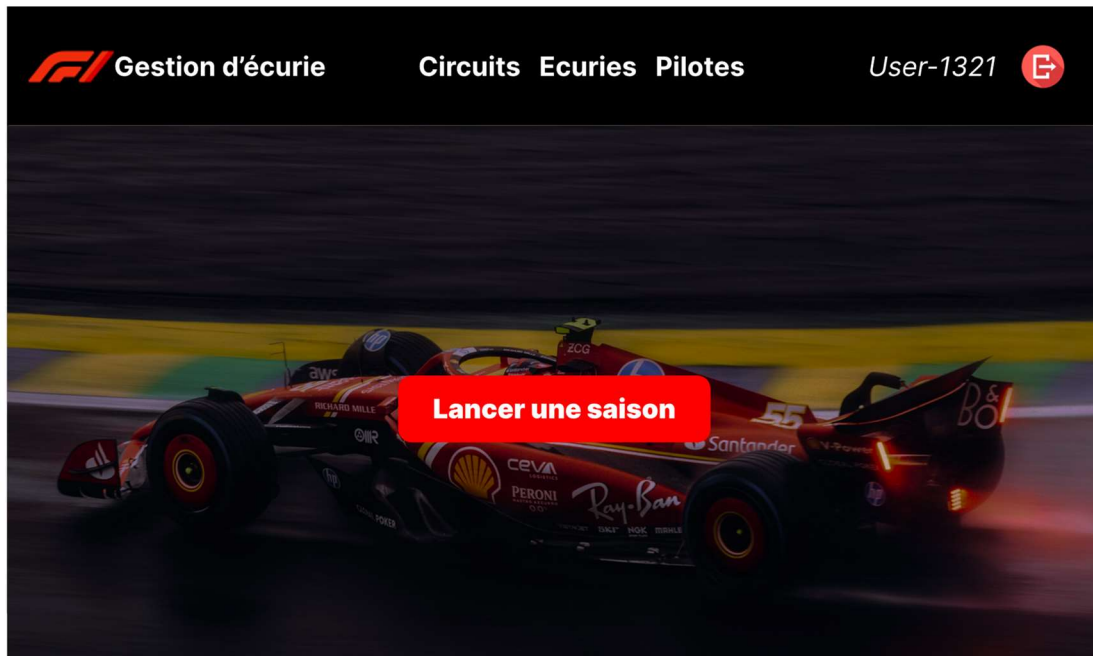


Figure 5: Maquette page principale

Sur la barre de navigation, la section ou était présent la zone « login/logout » dans le zoning contiendra le nom de l'utilisateur connecté ainsi qu'un bouton de déconnexion qui le reverra sur la page de connexion.

2.3.2.2 Liste des pilotes

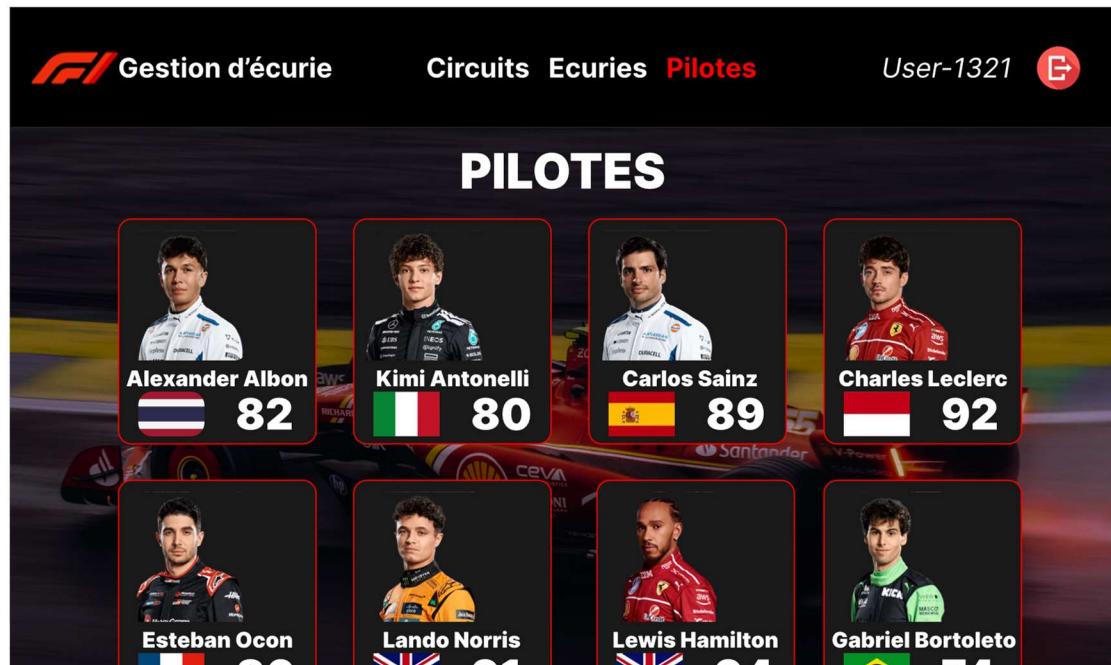


Figure 6: Maquette liste pilotes

Les zones présentes sur la maquette se rapprochent assez des zonings mais apportent plus de précision notamment au niveau de container des pilotes. Leurs noms seront affichés ainsi qu'une image, un rating et leur nationalité. Un administrateur aura une vue un peu différente de celle d'un simple utilisateur. Il verra affiché un bouton à côté de chaque pilote afin de pouvoir modifier ses informations. Un bouton redirigeant sur un formulaire de création de pilote sera aussi accessible uniquement aux administrateurs.

2.3.2.3 Liste des écuries

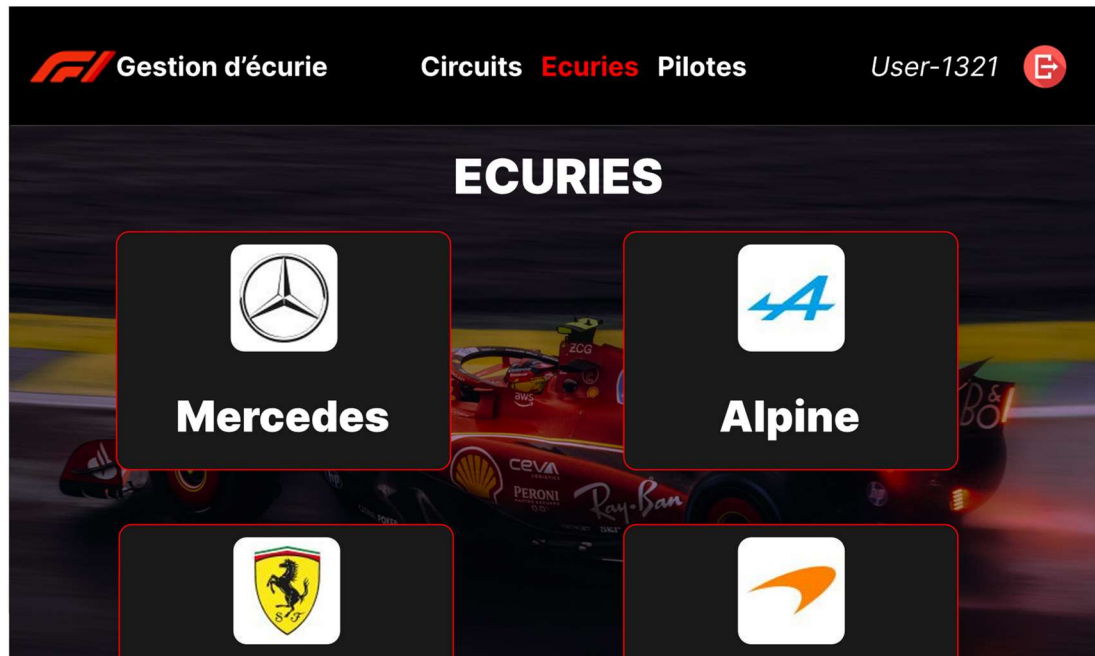


Figure 7: Maquette liste écuries

La liste des écuries reprends aussi les zonings avec la même fonctionnalité de connexion dans la barre de navigation. Un administrateur aura, de même, accès à un bouton pour modifier ou supprimer une écurie ainsi qu'un bouton qui redirige vers un formulaire de création d'écurie.

2.3.2.4 Classement après course



Figure 8: Maquette classement après-course

Après chaque course, un classement sera affiché. L'utilisateur pourra basculer entre le classement des pilotes et le classement des constructeurs. Les pilotes ainsi que l'écurie que l'utilisateur contrôle seront marqué d'une couleur différente afin de pouvoir les différencier plus facilement. Une image avec la course actuelle et la course suivant seront aussi affichée. Un compteur de course pourra aussi être affiché afin de donner plus d'informations utiles à l'utilisateur.

2.3.3 Modèle de base de données

Dans ce chapitre se trouvent les divers schémas de base de données du projet.

Les schémas ont été fait sur db main.

Pour l'implémentation de la base de données, j'ai choisi d'utiliser sqllite, un SGBDR léger et simple d'implémentation sans serveur. La base de données est stockée dans un fichier .db placé dans l'arborescence du projet, dans le dossier `/server/dbFiles`.

2.3.3.1 MCD

Voici le modèle conceptuel de données utilisé dans ce projet.

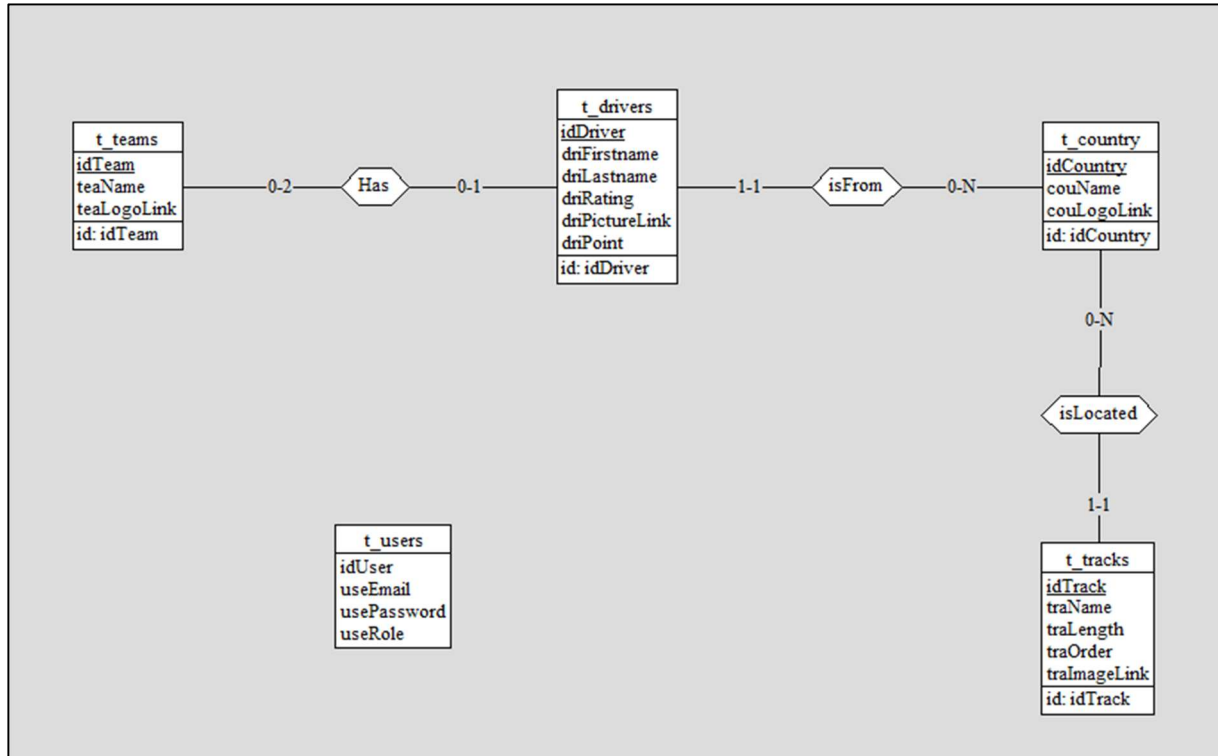


Figure 9 : Schéma MCD

- L'entité **t_teams** avec comme attributs un nom et lien du logo de l'écurie.
- L'entité **t_drivers** avec comme attributs le nom / prénom du pilote, le rating du pilote sous forme d'entier, le lien de la photo du pilote ainsi que le nombre de point du pilote.
- L'entité **t_tracks** avec comme attributs le nom du circuit, sa longueur, son ordre de passage dans la saison ainsi que l'image du tracé du circuit.
- L'entité **t_users** avec comme attribut l'adresse email de l'utilisateur qui sera l'identifiant naturel lors de la connexion au site, son mot de passe et son rôle.

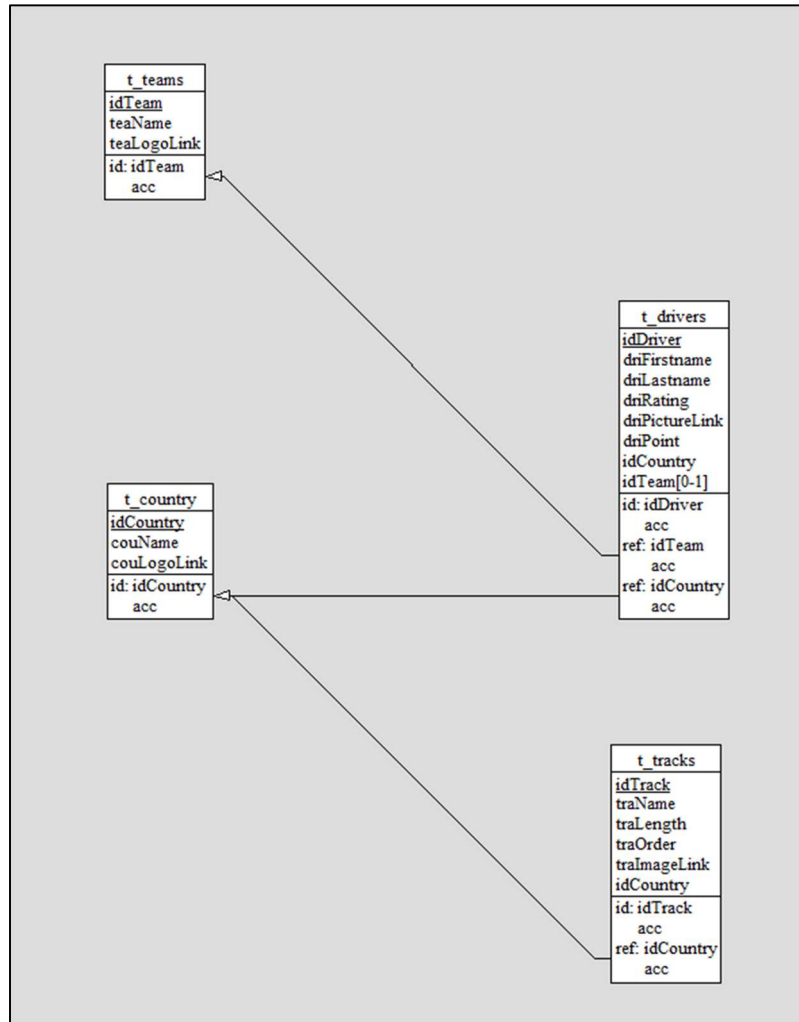
2.3.3.2 MLD

Figure 10 : Schéma MLD

Une équipe contient entre 0 et 2 pilotes maximum. Chaque pilote sera associé à une équipe, la clé étrangère *fkTeam* du pilote sera soit l'index d'une équipe, soit *null* au cas où il n'y pas pas assez d'équipe pour tous les pilotes.

La clé étrangère *fkCountry* est obligatoire pour un pilote ou un circuit.

2.3.3.3 MPD

Le modèle physique correspond à la structure finale de la base de données dans sqlite. On y retrouve directement les types de chaque champs ainsi que s'ils sont uniques ou non-nuls.

Tables (6)

Nom	Type	Schéma
sqlite_sequence		CREATE TABLE sqlite_sequence(name,seq)
name	TEXT	"name" TEXT
seq	TEXT	"seq" TEXT
t_country		CREATE TABLE "t_country" ("idCountry" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT UNIQUE, "couName" TEXT NOT NULL UNIQUE, "couLogoLink" TEXT NOT NULL UNIQUE)
idCountry	INTEGER	"idCountry" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT UNIQUE
couName	TEXT	"couName" TEXT NOT NULL UNIQUE
couLogoLink	TEXT	"couLogoLink" TEXT NOT NULL UNIQUE
t_drivers		CREATE TABLE "t_drivers" ("idDriver" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT UNIQUE, "driFirstname" TEXT NOT NULL, "driLastname" TEXT NOT NULL, "driRating" INTEGER NOT NULL, "driPictureLink" TEXT NOT NULL, "driPoint" INTEGER, "fkCountry" INTEGER NOT NULL, "fkTeam" INTEGER, FOREIGN KEY("fkCountry") REFERENCES "t_country"("idCountry"), FOREIGN KEY("fkTeam") REFERENCES "t_teams"("idTeam"))
idDriver	INTEGER	"idDriver" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT UNIQUE
driFirstname	TEXT	"driFirstname" TEXT NOT NULL
driLastname	TEXT	"driLastname" TEXT NOT NULL
driRating	INTEGER	"driRating" INTEGER NOT NULL
driPictureLink	TEXT	"driPictureLink" TEXT NOT NULL
driPoint	INTEGER	"driPoint" INTEGER
fkCountry	INTEGER	"fkCountry" INTEGER NOT NULL
fkTeam	INTEGER	"fkTeam" INTEGER
t_teams		CREATE TABLE "t_teams" ("idTeam" INTEGER UNIQUE, "teaName" TEXT NOT NULL UNIQUE, "teaLogoLink" TEXT NOT NULL, PRIMARY KEY("idTeam"))
idTeam	INTEGER	"idTeam" INTEGER UNIQUE
teaName	TEXT	"teaName" TEXT NOT NULL UNIQUE
teaLogoLink	TEXT	"teaLogoLink" TEXT NOT NULL
t_tracks		CREATE TABLE "t_tracks" ("idTrack" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT UNIQUE, "traName" TEXT NOT NULL UNIQUE, "traLength" INTEGER NOT NULL, "traOrder" INTEGER NOT NULL, "traImageLink" TEXT NOT NULL, "fkCountry" INTEGER NOT NULL)
idTrack	INTEGER	"idTrack" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT UNIQUE
traName	TEXT	"traName" TEXT NOT NULL UNIQUE
traLength	INTEGER	"traLength" INTEGER NOT NULL
traOrder	INTEGER	"traOrder" INTEGER NOT NULL
traImageLink	TEXT	"traImageLink" TEXT NOT NULL
fkCountry	INTEGER	"fkCountry" INTEGER NOT NULL
t_users		CREATE TABLE "t_users" ("idUser" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT UNIQUE, "useEmail" TEXT NOT NULL UNIQUE, "usePassword" TEXT NOT NULL, "useRole" TEXT NOT NULL)
idUser	INTEGER	"idUser" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT UNIQUE
useEmail	TEXT	"useEmail" TEXT NOT NULL UNIQUE
usePassword	TEXT	"usePassword" TEXT NOT NULL
useRole	TEXT	"useRole" TEXT NOT NULL

Figure 11 : MPD

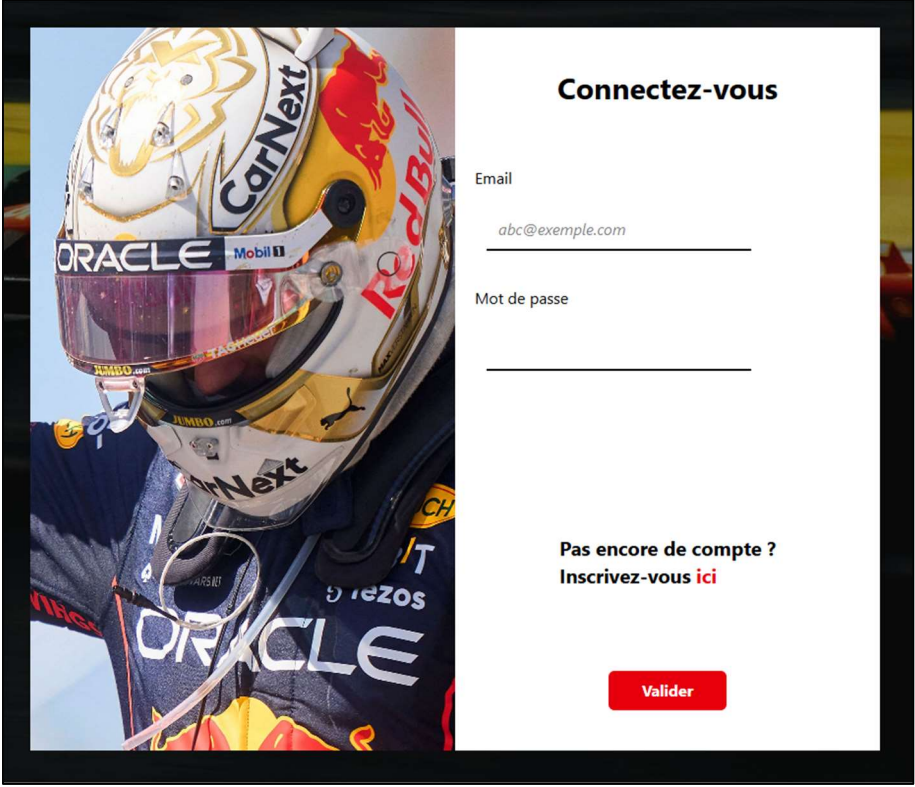
- Chaque pays possède un nom et un drapeau non-nul et unique.
- Les champs du pilotes autres que *fkCountry* et *driPoint* sont non-nuls et doivent donc contenir une valeur.
- Pour les équipes, tous les champs sont requis et le nom de l'écurie est unique.
- Similaire pour les circuits. Tous les champs sont non-nuls et le nom du circuit est unique.
- Chaque utilisateur doit posséder une adresse email, un mot de passe et un rôle. L'adresse email, étant l'identifiant naturel, doit être unique.

Même si logiquement, les liens d'images devraient être unique, ce n'est pas le cas car si, lors de la création d'un pilote par exemple, l'administrateur n'entre pas de photo, une photo par défaut est utilisée à la place et donc son lien *defaultDriverPic.png* est stocké dans la base de données. Il est donc possible que des pilotes aient la même photo. Cette logique est applicable aux circuits et aux équipes.

2.3.4 Analyse fonctionnelle

2.3.4.1 Connexion de l'utilisateur

L'utilisateur devra être identifié pour pouvoir profiter des fonctionnalités de l'application. Il aura accès à un formulaire de connexion où il devra entrer son email ainsi que son mot de passe. En cas d'identifiants incorrects, un message d'erreur averti l'utilisateur et une petite animation est affichée pour améliorer l'UX.



Connectez-vous

Email
abc@exemple.com

Mot de passe

Pas encore de compte ?
Inscrivez-vous **ici**

Valider

Figure 12 : Formulaire de connexion

2.3.4.2 Ajout pilote / écurie / circuit

Seul un administrateur aura accès à ces fonctionnalités.

L'utilisateur pourra ajouter un pilote, une écurie ou un circuit qui seront ensuite sélectionnable dans le jeu. Pour le pilote, par exemple, l'utilisateur entrera son nom, prénom, rating et sa nationalité via un formulaire. Une fois le formulaire validé, le pilote est ajouté à la base de données et est donc affiché dans la liste des pilotes.

Ajout de pilote

Prénom du pilote *

Max

Nom du pilote *

Verstappen

Rating du pilote *

98

Nationalité du pilote *

Nationalité

Image du pilote

Choisir un fichier Aucun fichier choisi

Valider

Figure 13 : Formulaire d'ajout de pilote

2.3.4.3 Modification pilote / écurie / circuit

Seul un administrateur aura accès à ces fonctionnalités.

L'utilisateur pourra modifier un pilote, une écurie ou un circuit. Les modifications seront prises en compte lorsqu'un joueur commencera sa partie. Les administrateurs ont accès à une icône de modification ou de suppression à côté de chaque pilote / écurie / circuit. L'icône de modification fait apparaître un formulaire similaire à celui de création à la différence que les champs sont déjà préremplis.

2.3.4.4 Jeu

La fonctionnalité de jeu est la fonctionnalité principale de l'application. Elle permet à l'utilisateur de sélectionner une écurie et 2 pilotes puis simule une saison avec contenant plusieurs courses. Entre chaque course, le classement général des pilotes et des constructeurs sera affiché pour que le joueur garde un œil sur le déroulement de la saison et de son équipe.

2.4 Stratégie de test

Différents tests seront effectués durant ce projet. La stratégie de test adoptée est la suivante :

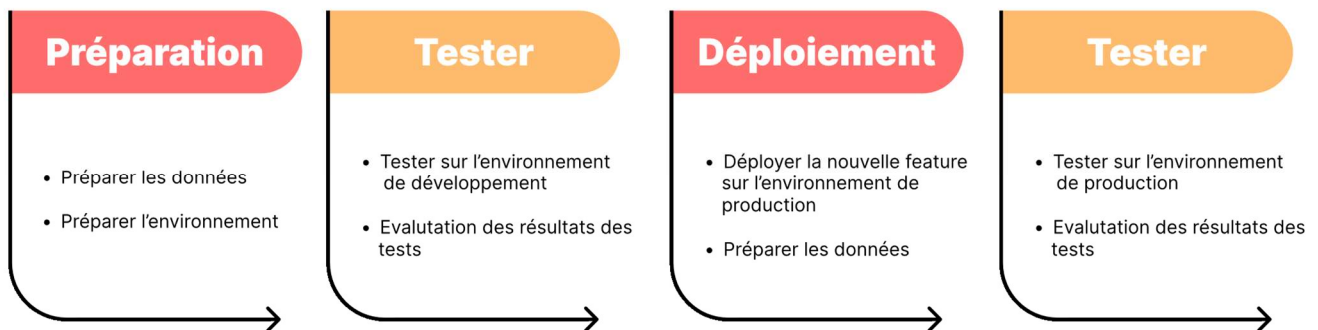


Figure 14 : Schéma de stratégie de test

Lorsqu'une nouvelle fonctionnalité a fini d'être développée, nous allons d'abord être sûr que les données requises sont prêtes. Nous allons aussi nous assurer que notre environnement est aussi prêt (c'est-à-dire que notre environnement est démarré avec la dernière fonctionnalité disponible). Je vais d'abord effectuer la liste des tests pour cette fonctionnalité sur l'environnement de développement. Puis les tests seront évalués selon les résultats attendus. Si tous les tests passent. La fonctionnalité sera déployée sur l'environnement de production. Si elle ne passe pas, une analyse des bugs sera faite afin de connaître ce qu'il faut corriger. Une fois que la *feature* est déployée sur l'environnement de production. Etant donné que cet environnement est différent de celui de développement, je vais réeffectuer la liste de test et réévaluer les résultats. Si tout est bon, la fonctionnalité sera déclarée comme « passée ». Sinon il faudra analyser l'erreur. Il faudra tout de même faire attention à ce qu'une nouvelle *feature* n'en casse pas une autre. Refaire rapidement les tests des autres *features* présentes sur le site sera une chose à faire.

Les tests seront effectués par moi-même au cours du projet

Je suis conscient qu'effectuer des tests sur l'environnement de production n'est pas l'idéal. Il serait préférable d'avoir un environnement de test non accessible aux utilisateurs et similaire à l'environnement de production dédié uniquement aux différents tests.

2.5 Risques techniques

Le risque principal est que j'ai appris React et ViteJs en autodidacte et en peu de temps de pratique. Je n'ai donc pas forcément les meilleurs pratiques lors de la réalisation et les risques d'être bloqué est plus élevée.

Pour réduire ces risques, j'ai, en amont de ce TPI, effectué un projet afin de me familiariser avec React, de gagner en aisance et apprendre un maximum son fonctionnement et sa logique afin de minimiser les moments bloqués durant le projet ainsi baisser les pertes de temps suite à ces problèmes.

Autre risque lié à l'environnement de production est que je n'ai pas de droit administrateur au serveur web de l'ETML. En cas de problème, je n'ai donc aucun droit et devrait donc demander à un administrateur ce qui peut entraîner des retards sur le projet.

3 Réalisation

3.1 Point de design spécifique

3.1.1 Gestion de l'authentification

Pour la gestion de l'authentification, j'ai choisi d'utiliser le modèle *RBAC*⁷. C'est-à-dire que chaque utilisateur a un rôle attribué et que les fonctionnalités ainsi sont accessible dépendamment du rôle de l'utilisateur.

L'idée du système d'authentification implémenté est d'avoir l'utilisateur connecté stocké dans le *local storage* du navigateur. Cela permettra à l'utilisateur de toujours être connecté même après avoir fermé l'onglet. L'application stocke donc un objet *currentUser* contenant l'identifiant naturel de l'utilisateur (son nom d'utilisateur) ainsi que son rôle. Le fait d'avoir l'utilisateur stocké permet d'éviter de devoir faire des requêtes à la base de données à chaque fois qu'il change de page et qu'on doit récupérer son rôle.

La logique d'authentification de l'application peut se représenter sous cette forme :

⁷ Role-Based Access Control

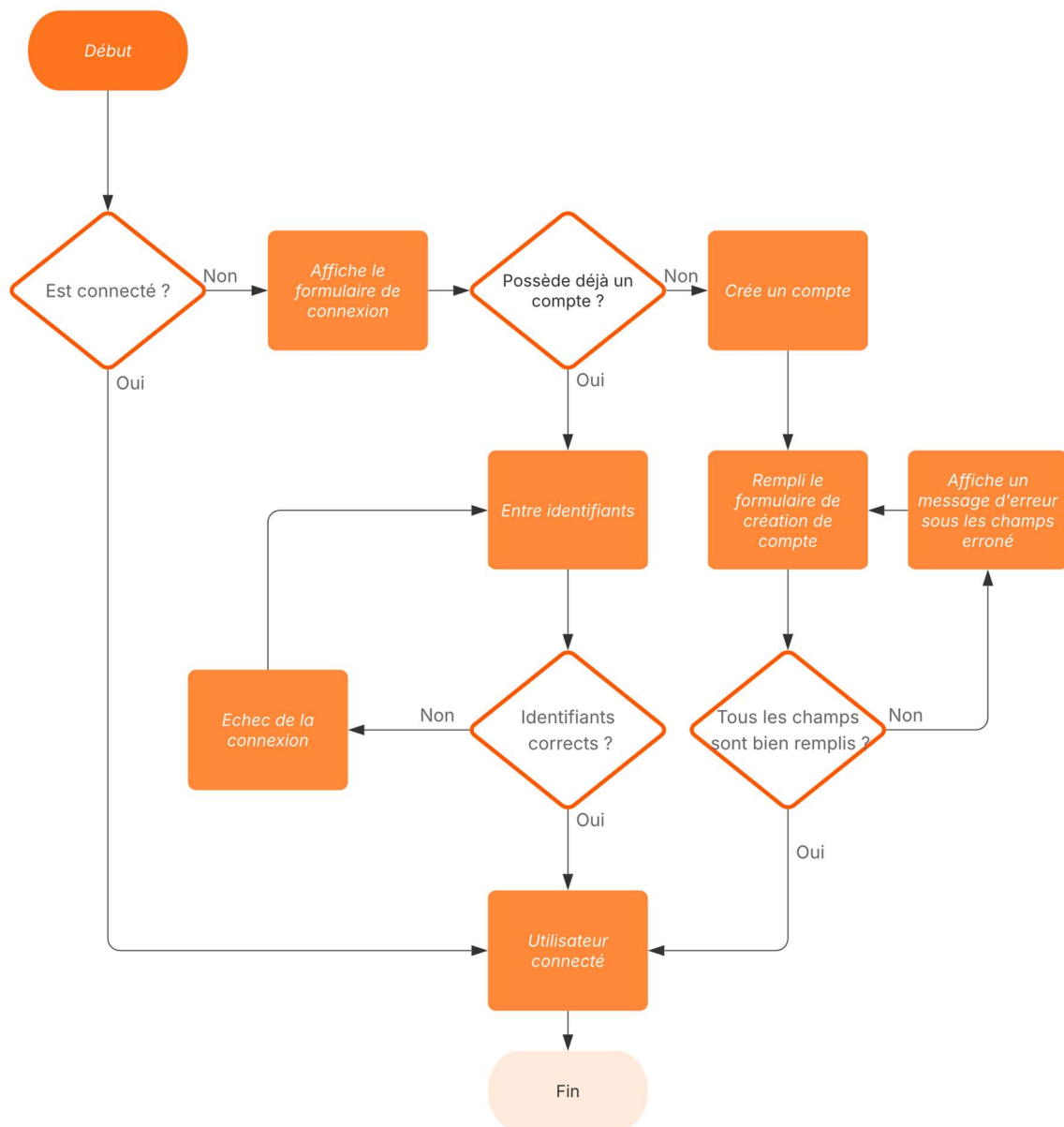


Figure 15: Logigramme authentification

- L'application va d'abord vérifier dans le *local storage* si un objet utilisateur est stocké. Si c'est le cas, alors c'est que l'utilisateur s'était connecté auparavant et donc l'application lui affiche la page principale.
- S'il n'est pas connecté, l'application lui affiche la page de login avec un formulaire de connexion.

- Une fois que l'utilisateur a entré ses identifiants et validé, une requête à la base de données est faite et vérifie si les identifiants sont corrects.
 - o Si c'est le cas, l'application stocke le nom d'utilisateur et son rôle dans un objet dans le *local storage* puis lui affiche la page principale.
 - o Si ce n'est pas le cas, un message d'erreur est affiché.

Mon cahier des charges ne demande pas de gestion de création de compte. Le schéma montre donc uniquement comment la création d'utilisateur serait gérée mais elle n'a pas été implémentée.

Pour cette gestion d'authentification, j'ai dû mettre en place un contexte d'authentification. Ce contexte possède plusieurs fonctions qui seront utilisées dans la gestion de l'authentification.

3.1.1.1 GetUser

```
4  export async function getUser() {  
5  
6      const authToken = localStorage.getItem("userToken")  
7  
8      if (authToken) {  
9  
10         const user = JSON.parse(atob(authToken.split('.')[1]));  
11  
12         return [200, { user }] as const;  
13     }  
14  
15     return [401, { authToken: null, user: null }] as const;  
16 }
```

Figure 16 : Fonction GetUser()

La fonction *GetUser* permet de vérifier s'il y a un token d'utilisateur stocké dans le *local storage* et de retourner le contenu de l'objet dans une variable (*currentUser*) qui sera

utilisée lors de la vérification des routes, etc... Si rien n'est trouvé, alors l'application définit l'utilisateur comme déconnecté et met à jour les différentes variables.

3.1.1.2 HandleLogin

La fonction *HandleLogin* va, comme l'indique son nom, gérer la connexion. Elle va d'abord envoyer une requête POST à la base de données pour récupérer les données de l'utilisateur correspondant au nom d'utilisateur entré dans le formulaire. Elle va récupérer le mots de passe hashé stocké dans la base de données puis va comparer avec le mot de passe entré dans le formulaire. Pour hasher le mot de passe et pour la comparaison, j'utilise la dépendance « *bcrypt* ».

Si les mots de passes correspondent, les données récupérées (le nom d'utilisateur et son rôle) puis sont stockées dans l'objet *currentUser*.

Si l'utilisateur rafraîchi la page, l'objet n'est pas sauvegardé. Pour régler ce problème, je vais stocker l'objet de l'utilisateur dans le local storage. Au début je pensais simplement laisser l'objet comme ça dans le local storage. Le problème était que n'importe quel utilisateur peut modifier le contenu de cet objet et donc peut se donner les droits administrateur. J'ai donc décidé de chiffrer l'objet en utilisant un JWT⁸. Avec ceci, l'utilisateur ne peut pas modifier son rôle et donc ne peut pas passer d'un utilisateur simple à un administrateur.

⁸ Json Web Token

```
62  async function handleLogin(email: string, password: string) {
63      try {
64          const response = await login(email, password);
65          const authToken = response[1].token;
66
67          setAuthToken(authToken);
68          setIsUserConnected(true);
69          localStorage.setItem('userToken', authToken);
70          try {
71              const decoded = JSON.parse(atob(authToken.split('.')[1]));
72              setCurrentUser({
73                  id: decoded.id,
74                  login: decoded.email,
75                  role: decoded.role,
76              });
77          } catch (err) {
78              console.error("Erreur décodage token :", err);
79          }
80
81          return null;
82      } catch {
83          setAuthToken(null);
84          setCurrentUser(null);
85          setIsUserConnected(false);
86          return "erreur";
87      }
88  }
```

Figure 17 : Fonction handleLogin()

L'utilisation du Token se fait suivant la logique suivante :

Une clé secrète est stockée dans le fichier .env dans le côté serveur. Lorsque le backend fait une requête à la base de données, il transforme l'objet reçu en retour en JWT pour le renvoyer au frontend qui s'occupe de le stocker dans le local storage. Le frontend va ensuite, lorsqu'il en a besoin, déchiffrer le token pour récupérer les informations de l'utilisateur.

```
44 useEffect(() => {
45     if (authToken) {
46         try {
47             const decoded = JSON.parse(atob(authToken.split('.')[1]));
48             setUserRole(decoded.role);
49         } catch (e) {
50             console.error("Erreur lors du décodage du token :", e);
51         }
52     }
53 }, [authToken]);
```

Figure 18 : Déchiffrement JWT

Si le token est manuellement modifié, l'utilisateur est automatiquement déconnecté afin de régénérer un nouveau token valide.

Voici notre objet utilisateur qui sera stocké :

```
{
  "id": 2,
  "email": "admin",
  "role": "admin",
  "iat": 1747833318,
  "exp": 1747840518
}
```

Figure 19 : Objet Utilisateur après déchiffrement

Et voici à quoi cela ressemble au format JWT

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MiwiZWlhaWwIjHhZG1pbIiIsInVjbGUOIjHhZG1pbIiIsImhhbmQCI6MTc0NzgzMzIxMzcwZXhwIjojOzQ3ODQwNTE4fQ.XoFc4xsgIJavF7BhVVF-FgkItWGPXOG7 jB0z04tUUM

Figure 20 : Exemple JWT

3.1.1.3 HandleLogout

La gestion de déconnexion est plutôt simple. Elle va faire appel en premier lieu à la fonction *getUser* dont j'ai décrit le fonctionnement tout à l'heure. Elle va donc récupérer le token de l'utilisateur actuellement connecté puis va le supprimer. Comme la fonction *getUser* définit l'état connecté/déconnecté selon s'il trouve ou non un token utilisateur dans le *local storage*, il va donc donner désormais l'état déconnecté.

3.1.2 Gestion des routes frontend

En react, les différentes routes représentent les pages ou les composants affichés à l'écran selon l'URL de l'application. Par exemple si l'URL */mainPage* est actif, la page affichée sera la page principale. Ces distinctions se font grâce à la dépendance « *react-dom-router* ». C'est aussi avec ceci que la gestion des accès aux pages se feront. J'ai créé un composant *ProtectedRoute* qui prend en *props* ⁹un tableau des rôles qui seront autorisés à accéder à cette route. Si un utilisateur qui n'a pas le rôle permettant d'accéder à cette route essaie de s'y rendre en rentrant directement le lien de la page dans l'URL, alors il sera tout de suite redirigé à la page de défaut, dans notre cas la page principale. Dans ce projet, actuellement, cette redirection n'a pas grande utilité car il n'y a pas de page interdite à un type d'utilisateur, que des fonctionnalités affichées ou non selon le type. Cependant, avoir ceci d'implémenter permet un meilleur scalabilité de l'application et permettra d'ajouter de nouvelles fonctionnalités et de gérer ses accès beaucoup plus facilement. Cela permettra aussi de rajouter des rôles si besoin comme, par exemple, un utilisateur de type *pilote-reader* qui aura le droit de voir que la page des pilotes. C'est aussi dans ce composant que l'on redirige un utilisateur non-identifié à la page de login. Si l'objet *currentUser* est *null*, alors cela signifie qu'il n'y a pas d'objet user dans le *local storage*. Comme vu précédemment, cela veut donc dire que l'utilisateur est déconnecté et sera donc automatiquement redirigé au formulaire de connexion. Il ne lui sera pas possible d'aller sur d'autre page.

⁹ Argument passé à un composant

Finalement, voici à quoi ressemble une définition de route :

```
25   <Route path="/mainPage" element={  
26     <ProtectedRoute allowedRoles={['admin', 'user']}>  
27       <MainPage />  
28     </ProtectedRoute>} />
```

Figure 21: Exemple d'une route

On retrouve bien là l'URL de la route, le composant de protection de route qui a comme *props* le tableau des rôles autorisés à accéder au contenu et en enfant de ce composant le composant de la page principale.

Une route importante à définir est celle par défaut. Si par exemple l'utilisateur entre dans l'URL la route */cettePageNExistePas* alors il sera redirigé sur la page principale. La protection de routes fera aussi effet sur cette redirection.

3.1.3 Gestion des pilotes / écuries / circuits

La gestion des pilotes, des écuries et des circuits sont 3 points techniques relativement similaires. J'ai donc décidé de les regrouper en un seul chapitre et d'expliquer son fonctionnement en prenant la gestion des pilotes comme exemple.

Les seules différences seront au niveau de l'affichage des *cards* et du nombre de champs dans les formulaires de création et de modification. Mais les fonctionnalités d'ajout, de modification et de suppression ont le même fonctionnement.

3.1.3.1 Composition de la page

Le point positif avec React c'est que la composition des pages avec ses composants ressemble beaucoup au zoning de la page. J'ai donc repris globalement le zoning réalisé en début de projet pour gérer la composition de la page. Finalement, la composition de la page ressemble à ça :

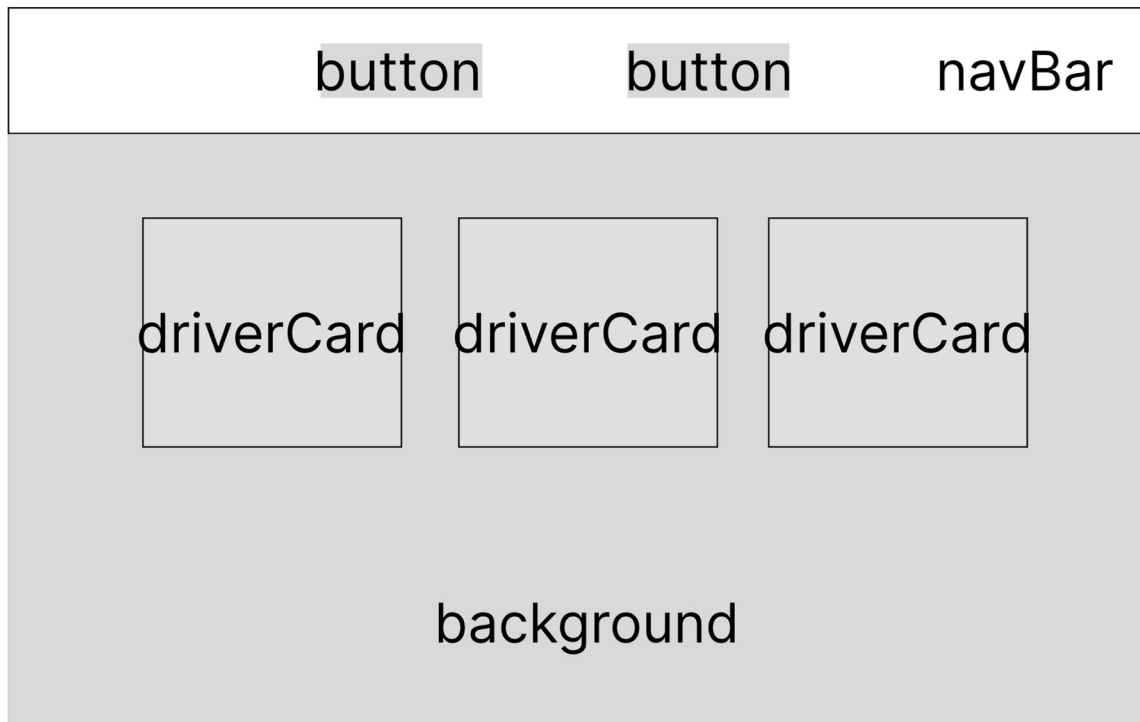


Figure 22: Composition composants liste pilotes

Un autre point positif de React est son architecture basée sur les composants. Cela permet de réaliser un seul composant de type *card*, et de le réutiliser autant de fois que nécessaire. Ainsi, qu'on ait 1 ou 45 pilotes dans notre base de données, on a juste à reprendre les données et les parcourir avec une boucle (*.map*) pour afficher tous nos pilotes.

Lorsque l'utilisateur accède à la page */drivers*, l'application envoie une requête http à l'URL */api/drivers* (donc le côté backend). Le *controller* associé à cette route appelle la fonction *getAllDriver*. La fonction va ensuite faire une requête SQL pour récupérer la liste des pilotes. Le *controller* va retourner le résultat de la requête au frontend.

On retrouve ici le fonctionnement *client-server* décrit quelques chapitres auparavant expliquant comment le lien entre *backend* et *frontend* se fait. On a donc maintenant un tableau de pilote qu'on va pouvoir afficher.

Dans le frontend, j'ai ajouté un fichier « interface ». Ce fichier contient la déclaration des types personnalisés qui seront utilisés dans l'application. Par exemple, *driverInterface* décrit la structure d'un objet « pilote » :

```
16 export type driverInterface = {  
17     id: number;  
18     firstname: string;  
19     lastname: string;  
20     rating: number;  
21     nationality: string;  
22     team: string;  
23     pictureLink: string;  
24     countryFlag: string;  
25 }
```

Figure 23: Interface d'un objet pilote

Cela permet d'avoir un développement plus fiable et plus assisté (proposition des paramètres par VSC¹⁰).

L'application va d'abord afficher le composant *navBar* avec comme onglet actif « Pilotes » donc sera mis en rouge.

Elle va ensuite récupérer le rôle de l'utilisateur actuellement connecté. Si son rôle est « admin » alors l'application va y afficher les boutons de création de pilotes et de modification de pilotes.

Pour l'affichage des *cards*, on va parcourir le contenu du tableau de pilotes et, pour chaque pilote, afficher un composant avec le pilote comme *props* :

¹⁰ Visual Studio Code


```
90 {driverList.map((driver) => (  
91   <>  
92     <div className="w-fit h-fit border-2 rounded-2xl border-red-700  
93       <DriverCard driver={driver} onEdit={handleEditDriver} />  
94     </div>  
95   </>  
96 )>}
```

Figure 24: Affichage de la liste des pilotes

3.1.3.2 Ajout d'un pilote

Un bouton visible uniquement par les administrateurs permet de faire apparaître un formulaire de création de pilote. Le formulaire contient des inputs pour entrer le nom, le prénom ainsi que le rating du pilote, une liste déroulante pour sélectionner un pays et un input pour sélectionner un photo. Cette dernière partie n'a pas été implémentée. Cependant, pour chaque nouveau pilote, une image de profil par défaut est insérée.

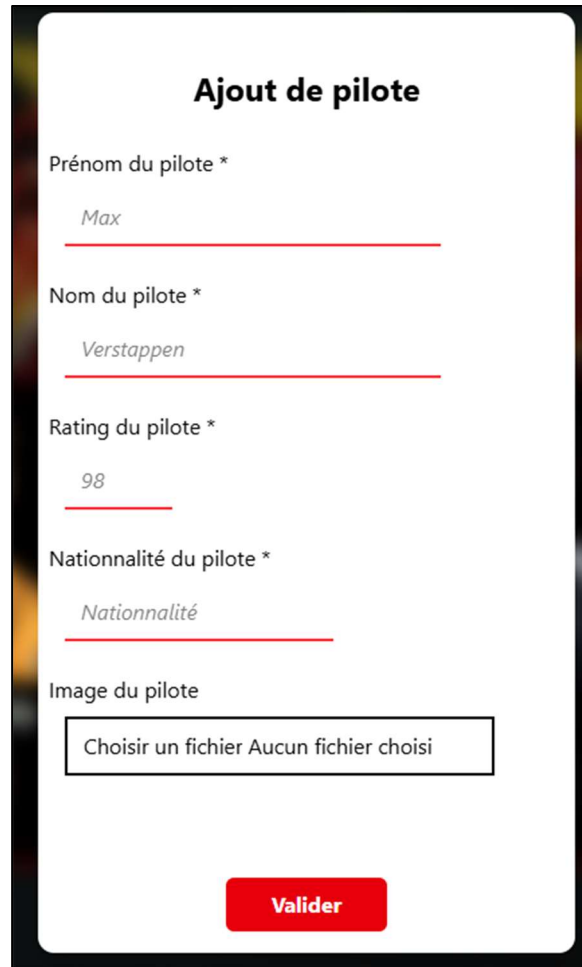


Figure 25 : Formulaire d'ajout de pilote

Une fois que l'administrateur valide chacun des champs du formulaire, l'application va vérifier si les champs obligatoires (nom, prénom, rating, nationalité) sont bien remplis, si c'est le cas, alors il envoie une requête au backend qui, lui, va ajouter le pilote dans la base de données. Si tous les champs obligatoires ne sont pas remplis, alors un message d'erreur apparaît et rien ne se passe.

3.1.3.3 Modification d'un pilote

Lorsqu'un administrateur se rend sur la page des pilotes, un logo de modification apparaît à côté de chaque pilote. S'il clique dessus, alors un formulaire similaire à celui de création apparaît à la seule différence que les champs sont déjà préremplis avec les données du pilote que l'administrateur veut modifier.

La vérification des données est la même que pour le formulaire d'ajout et la modification des données dans la db par le backend est gérée de la même façon.

3.1.3.4 Suppression d'un pilote

Lorsqu'un administrateur se rend sur la page des pilotes, un logo de suppression apparaît à côté de chaque pilote. Lorsqu'il clique dessus une requête est alors envoyée et le pilote est alors supprimé de la base de données.

3.1.4 Gestion de jeu

Le jeu est la fonctionnalité principale de l'application. Pour démarrer une partie, il faut d'abord être authentifié pour avoir accès à la page principale où se lance la partie. Pas besoin d'être administrateur, un simple utilisateur peut jouer.

Un bouton permettant de lancer la partie se trouve sur la page principale. Il s'y trouve aussi sur cette page un bouton pour reprendre une partie dans le cas où l'utilisateur avait déjà lancé une partie auparavant et que les données sont toujours sauvegardées dans le local storage.

Avant d'entrer dans les détails de chaque étape du jeu, voici un schéma décrivant la logique du lancement du jeu :

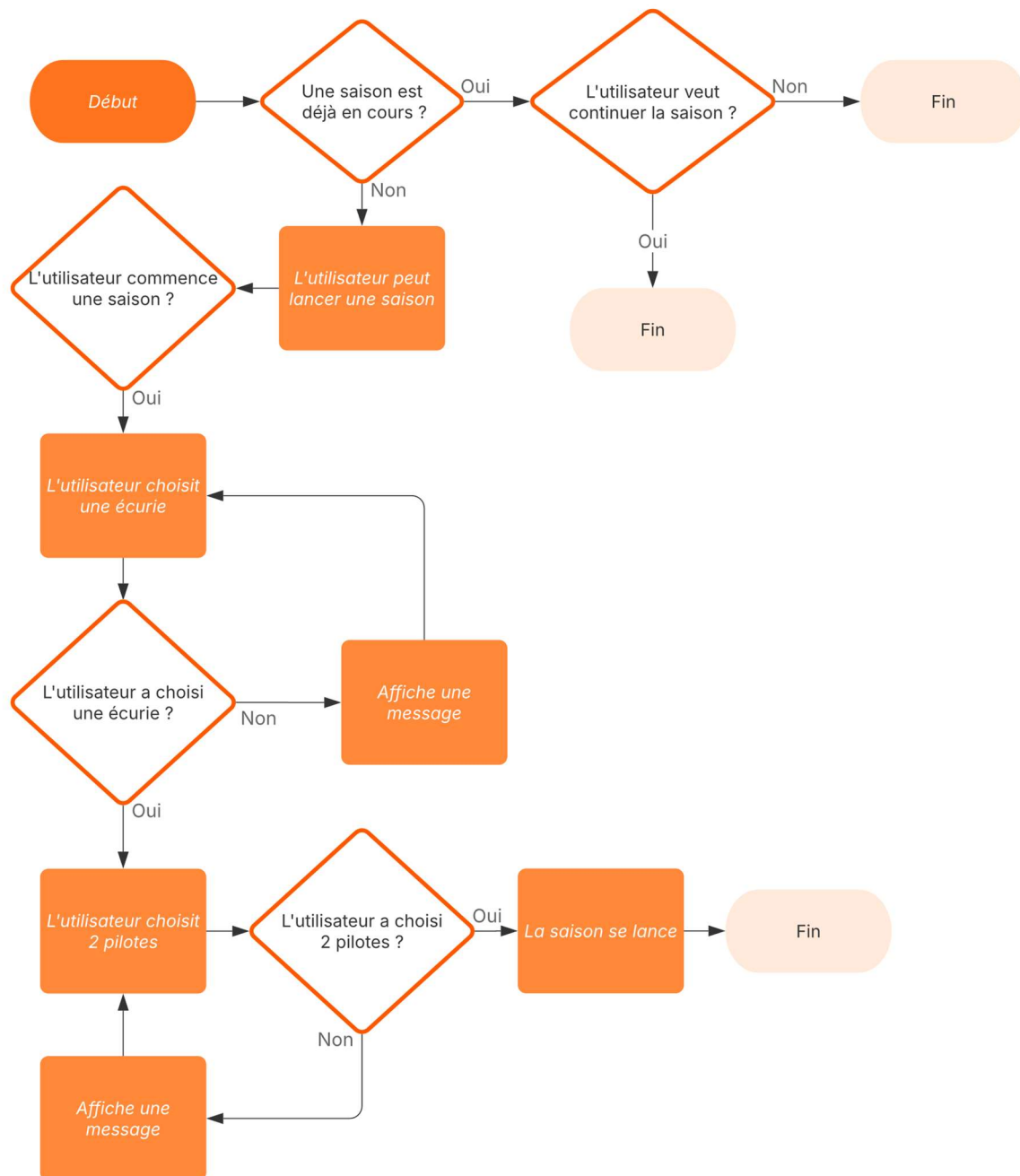


Figure 26 : Logigramme de lancement d'une saison

Le schéma sera expliqué au cours de prochains chapitres.

3.1.4.1 Choix de l'écurie et des pilotes

Lorsqu'un utilisateur choisi de démarrer une nouvelle saison, il accède à la page */game*. La première chose qui est affiché à l'utilisateur est la page de sélection de l'écurie. C'est une page ressemblant à la page des écuries. Le joueur choisi son écurie en cliquant dessus. L'écurie choisie est donc encerclée en rouge. Le joueur est obligé de choisir une et une seule écurie pour continuer.

Lorsque le joueur accède à cette page, l'application stocke une valeur *inGame* : *true* dans le local storage. Cela permet de savoir si l'utilisateur à déjà lancé une partie ou non.

Une fois l'écurie choisie, le joueur choisi ses 2 pilotes. Exactement 2 pilotes doivent être choisi pour que l'utilisateur puisse continuer. Pour gérer la sélection des pilotes, j'ai mis en place un système de *First In, First Out*. C'est-à-dire :

- Choisi pilote N°1 en cliquant sur un pilote
- Choisi pilote N°2 en cliquant sur un autre pilote
- Si sélectionne un 3^{ème} pilote, le pilote N°1 (qui est le pilote les plus « vieux » dans les pilotes sélectionnés) est remplacé par ce nouveau pilote

```
59  const handleDriverSelection = (driver: driverInterface) => {  
60  
61      //FIFO  
62  setSelectedDrivers((prev) => {  
63      if (prev.find(d => d.id === driver.id)) {  
64          return prev.filter(d => d.id !== driver.id);  
65      }  
66      if (prev.length < 2) {  
67          return [...prev, driver];  
68      }  
69      return [prev[1], driver];  
70  });  
71  };
```

Figure 27 : FIFO, sélection des pilotes

Si l'utilisateur clique sur un pilote déjà sélectionné, alors il est désélectionné.

Lorsque le joueur a choisi ses 2 pilotes, alors la saison peut commencer. Pour cela, l'application appelle la fonction *StartSeason()* qui appartient au contexte du jeu. C'est cette fonction aussi qui va répartir les pilotes restants dans les écuries de façon aléatoire.

Pour ça, elle va envoyer une requête au backend pour récupérer la liste des pilotes et des écuries. Elle va ensuite filtrer le tableau reçu afin d'enlever les pilotes choisis par le joueur. Ensuite, pour chaque écurie, on va vérifier si c'est l'écurie que le joueur a choisi, si c'est le cas alors on y ajoute les 2 pilotes sélectionnés, sinon, on va chercher 2 pilotes en avec un index aléatoire puis on modifie, dans la base de données, le champs *fkTeam* avec l'id de l'écurie dans laquelle il vient d'être mis.

Sachant qu'un administrateur peut ajouter des pilotes, il se peut qu'il y ait plus de pilotes que de places disponibles. Pour cela, les pilotes restants sont aussi modifiés et leur champ *fkTeam* est modifié avec la valeur *null*. Lorsque la saison commence, les pilotes ayant *null* comme valeur ne seront pas pris en compte.

```

82  const tempTeamArray: teamInterface[] = resTeams.data;
83  // ForEach team
84  tempTeamArray.forEach(team => {
85    // if the id matches the selected team id, edit the driver's team assigned
86    if (team.id === selectedTeam?.id) {
87      for (const driver of selectedDrivers) {
88        axios.post("/api/drivers/updateTeam", {
89          idDriver: driver.id,
90          idTeam: selectedTeam.id,
91        }).catch(err => {
92          console.error(`Erreur lors de l'affectation de ${driver.firstname}`, err);
93        });
94      }
95      return
96    }
97
98    const teamDrivers: driverInterface[] = [];
99    // 2 drivers per team
100   for (let i = 0; i < 2; i++) {
101     if (tempDriverArray.length === 0) break;
102     // select a random number, this will be the index of the driver to assign to the team
103     const randomIndex = Math.floor(Math.random() * tempDriverArray.length);
104     const selectedDriver = tempDriverArray[randomIndex];
105
106     axios.post("/api/drivers/updateTeam", {
107       idDriver: selectedDriver.id,
108       idTeam: team.id,
109     }).catch(err => {
110       console.error(`Erreur lors de l'affectation de ${selectedDriver.firstname}`, err);
111     });
112     teamDrivers.push(selectedDriver);
113     tempDriverArray.splice(randomIndex, 1);
114   }
115 });
116 // drivers that are leftover are set to null
117 axios.post("/api/drivers/updateTeam", {
118   idDriver: tempDriverArray[0].id,
119   idTeam: null,
120 }).catch(err => {
121   console.error(`Erreur lors de l'affectation de ${tempDriverArray[0].firstname}`, err);
122 });

```

Figure 28 : Distribution des pilotes dans les écuries

Le local storage est aussi mis à jour avec des nouvelles valeurs :

- **gameState : start** Cette variable permet de définir dans quelle état est la partie. Cela permettra de reprendre une partie là où l'utilisateur l'avait laissé.
- **gameSettings** : Cet item stock un objet qui contient l'écurie choisie par le joueur et les 2 pilotes choisis ainsi que le numéro de la course.

3.1.4.2 Simulation de course

La fonction de simulation de course est plutôt simple. Elle va d'abord effectuer une requête à la base de données pour récupérer les pilotes qui font partie d'une équipe et les mettre dans un tableau. Puis, pour chaque pilote présent dans ce tableau, va reprendre leur rating et va la modifier en y ajoutant entre -20 et 20. La fonction va ensuite choisir au minimum 2 nombre d'abandon qui auront lieu durant la course. Une boucle *for* avec comme nombre d'itération le nombre d'abandon. Pour chaque itération, un index est tiré aléatoirement, ce pilote est ajouté au tableau des abandons. Le tableau des résultats est ensuite ordonné selon le rating modifié des pilotes par ordre décroissant.

Les 10 premiers reçoivent des points selon la distribution suivante :

1 => 25 points ; 2 => 18 points ; 3 => 15 points ; 4 => 12 points ; 5 => 10 points ; 6 => 8 points ; 7 => 6 points ; 8 => 4 points ; 9 => 2 points ; 10 => 1 points

La base de données est donc mise à jour avec les nouveaux points des pilotes. Les tableau des résultats de la course sont retournés.


```
148   try {
149     const resDrivers = await axios.get("/api/drivers/getDriverInATeam");
150
151     const driverListRace: driverInterface[] = resDrivers.data;
152
153     driverListRace.forEach((driver: driverInterface) => {
154       driver.rating = driver.rating + (Math.floor(Math.random() * 41) - 20)
155     })
156
157     let nbDNF = Math.floor(Math.random() * driverListRace.length / 7) + 2;
158
159     let dnfDriver: driverInterface[] = []
160
161     for (let i = 0; i < nbDNF; i++) {
162       let dnfIndex = Math.floor(Math.random() * driverListRace.length)
163
164       dnfDriver.push(driverListRace[dnfIndex])
165       driverListRace.splice(dnfIndex, 1)
166
167     }
168     setDnfDriver(dnfDriver)
169     SetRaceResult(driverListRace.sort((a, b) => b.rating - a.rating))
170
171     for (let i = 0; i < pointDistribution.length; i++) {
172       driverListRace[i].point += pointDistribution[i]
173       await axios.put('/api/drivers/updatePoints',
174         {
175           idDriver: driverListRace[i].id,
176           points: driverListRace[i].point
177         }
178       )
179     }
180   }
181   const resultToSave = [driverListRace, dnfDriver]
182   localStorage.setItem("raceResult", JSON.stringify(resultToSave));
```

Figure 29 : Simulation de la course

3.1.4.3 Gestion des classements

Les classements affichés seront des composants réutilisables. Il y a 2 composants :

- driverLeaderboard : Le classement des pilotes
- teamLeaderboard : Le classement des constructeurs (des écuries)

Les 2 composants ont un fonctionnement similaire. La seule différence est au niveau de l'affichage du tableau, mais la logique est basiquement la même. Je vais donc prendre le classement des pilotes comme exemple pour expliquer son fonctionnement.

Les props de ce composant sont :

```
5  type Props = {  
6    |   isResult: boolean;  
7    |   driverList: driverInterface[] | null;  
8  };
```

Figure 30 : Props du composant du classement

isResult étant un booléen permettant de faire la différence entre le classement affiché après une course qui affichera uniquement la distribution des points ou celui qui affichera le classement de toutes les courses.

La liste des pilotes passée est triée dans l'ordre décroissant des points avant d'être affichés dans le tableau.

```
const { selectedDrivers } = useGame()  
const pointDistribution = [25, 18, 15, 12, 10, 8, 6, 4, 2, 1]  
let counter = 0  
if (!driverList) return null;  
  
const orderedDrivers = driverList  
  ? isResult  
    ? [...driverList]  
    : [...driverList].sort((a, b) => b.point - a.point)  
  : [];
```

Figure 31 : Tri des pilotes

Avant d'être affiché, une autre vérification est faite. Celle de savoir si le pilote qui sera affiché est celui sélectionné par le joueur. Si c'est le cas, ce pilote sera mis en évidence avec un fond rouge.

```

31 <tbody>
32   <tr className="text-start h-10 text-2xl italic font-bold □text-white">
33     <th>Pos</th>
34     <th>Nom du pilote</th>
35     <th>Ecurie</th>
36     <th>Points gagnés</th>
37   </tr>
38   {orderedDrivers.map((driver, index) => {
39     if (driver.point == null) driver.point = 0;
40
41     const isUserDriver = selectedDrivers.length >= 2 &&
42       (driver.id === selectedDrivers[0].id || driver.id === selectedDrivers[1].id);
43
44     return (
45       <tr key={driver.firstname} className={`□text-white text-xl font-medium h-10 ${isUserDriver ?
46         <td className="pl-10">{index + 1}</td>
47         <td className="pl-10">{driver.firstname + " " + driver.lastname}</td>
48         <td className="pl-10">{driver.team}</td>
49         <td className="pl-10">{isResult ? pointDistribution[counter++] || 0 : driver.point}</td>
50       </tr>
51     );
52   })}
53 </tbody>

```

Figure 32 : classement des pilotes

3.1.4.4 Fin de la saison

La fin de saison se fait une fois que toutes les courses sont finies. Un classement final alors apparaît. Pour cela, j'ai utilisé 1 seul composant qui affiche les pilotes et les écuries. Son fonctionnement est similaire aux simples classements vus précédemment mais le top 3 est affiché avec des cartes pour faire comme un podium et les pilotes / écurie sélectionnés sont mis en évidence.

CLASSEMENT FINAL

Classement des pilotes

Classement des constructeurs

#2 183 points



George Russell
Ferrari

#1 192 points



Oscar Piastri
Mercedes

#3 174 points



Max Verstappen
Mercedes

Pos	Nom du pilote	Ecurie	Points gagnés
4	Kimi Antonelli	Williams	160
5	Fernando Alonso	Red Bull Racing	154

[Retour au menu principal](#)

Pour la logique, il faut d'abord retirer le top 3 des pilotes / écuries pour les trier dans l'ordre décroissant des points de chacun.

```

61  const orderedDrivers = driverList
62  |   ? isResult
63  |   |   ? [...driverList]
64  |   |   : [...driverList].sort((a, b) => b.point - a.point)
65  |   : [];
66  const shortDriverList = orderedDrivers.slice(3, orderedDrivers.length - 1);
67  const shortTeamList = computedTeamList.slice(3, computedTeamList.length - 1);
68

```

2 onglets sont aussi affichés. Celui sélectionné a un fond rouge. Pour changer d'onglet et donc de classement, il suffit de cliquer sur l'onglet en gris. Cela nous permet de passer du classement des pilotes à celui des constructeurs sans aucun rafraichissement de la page.

3.2 Dossier de réalisation

3.2.1 Mise en place de l'environnement de travail

Le projet se trouve sur le repo Github. Dans le fichier *app* se trouve le code source de l'application. Dans le dossier *server* se trouve tout ce qui est backend. On y trouve les routes ainsi que les controllers. Le fichier de base de données se trouve dans le dossier *dbFiles*.

Pour mettre en place l'environnement de travail, il faut d'abord cloner le repo sur votre machine locale. Si ce n'est pas déjà fait, il faut installer NodeJs, installable depuis le site officiel de NodeJs et faire attention à installer npm avec. Pour vérifier si NodeJs est déjà installé, ouvrez une invite de commande et entrez :

```
Node -v
```

Une fois NodeJs installé, ouvrez une invite de commande depuis le dossier *app* puis exécutez la commande suivante :

```
npm install
```

Faite de même depuis le dossier *server*.

Cela installera les dépendances dont le projet a besoin.

Ensuite, il va falloir démarrer le backend et le frontend de l'application,

Pour démarrer le backend, il suffit d'ouvrir une invite de commande depuis le dossier *app* et d'y rentrer :

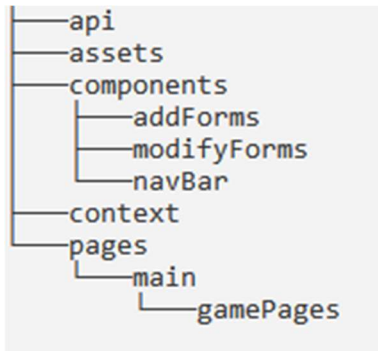
```
cd server
```

Puis

```
node index.js
```

Vous devriez avoir votre backend démarré sur <http://localhost:3040>

Dans le dossier `src` se trouve la partie frontend. Voilà à quoi ressemble son arborescence :



- Le dossier `api` qui contient les fonctions de connexion.
- Le dossier `components` qui contient tous les composants utilisés dans l'application
- Le dossier `context` qui contient les différents context comme le `gameContext`
- Le dossier `pages` qui contient toutes les pages utilisées dans le projet.

Pour démarrer le frontend, il suffit d'ouvrir une invite de commande depuis le dossier `app` et d'y entrer :

npm run dev

Dans l'invite de commande sera ensuite affiché le lien ainsi que le port sur lequel le frontend est démarré. Par exemple :

```
C:\Users\pv950qz\Desktop\ReactProject\GestionEcurieF1-TPI\app>npm run dev

> app@0.0.0 dev
> vite

VITE v6.3.5 ready in 765 ms

  Local:   http://localhost:5173/
  Network: use --host to expose
  press h + enter to show help
```

Ouvrez ensuite un navigateur web et entrez cette URL, vous devriez avoir votre environnement de travail prêt.

Le développement a été fait sur un PC sous OS Windows 10. Le développement de l'application s'est fait avec Visual Studio Code. La version de NodeJs utilisée est la version 22.14.0. Pour interagir avec ma base de données plus facilement. J'ai utilisé l'application DB Browser.

Pour l'environnement développement et de production, il y a 2 utilisateurs présents dans la base de données afin d'effectuer des tests. Ces utilisateurs sont :

Email : **user** / mot de passe : **pass**

Email : **admin** / mot de passe : **pass**

3.2.1.1 Dépendances utilisées

Voici la liste des dépendances principales et de leur version utilisées :

- ExpressJs : 4.20.0
- Cors : 2.8.5
- Dotenv : 16.5.0
- Sqlite3 : 5.1.7
- Tailwindcss : 4.1.5
- Axios : 1.9.0
- Bcrypt : 6.0.0
- Jsonwebtoken : 9.0.2
- React : 19.1.0
- React-router-dom : 7.6.0
- Typescript : 5.8.3
- Vite : 6.3.5

3.3 Description des tests effectués

Fonctionnalité	Procédure	Résultat attendu	Résultat	Commentaire
Ajout de pilote	Ajouter un pilote avec un nom et un prénom de type texte, un rating de type nombre et une nationalité.	Le pilote est créé et se retrouve dans la liste des pilotes.		La nationalité doit être choisie dans la liste déroulante
	Ajouter un pilote avec un nom contenant des chiffres.	Un message disant que le nom n'est pas correct et le pilote ne se crée pas.		Le pilote est quand même créé
	Ajouter un pilote avec une lettre dans le rating.	Il n'est pas possible d'y entrer des lettres		
	La fonctionnalité d'ajout est accessible uniquement par un administrateur.	Un utilisateur simple n'a pas le bouton d'ajout affiché sur son écran.		
Modification de pilote	Modifier un pilote avec un nom et un prénom de type texte, un rating de type nombre.	Seul le pilote visé par la modification est modifié.		
	Les données avant modifications du pilote se retrouvent dans les bons champs.			
	Modifier un pilote avec un nom contenant des chiffres.	Un message disant que le nom n'est pas correct et le pilote n'est pas modifié		Le pilote est quand même modifié
	Modifier un pilote avec une lettre dans le rating.	Il n'est pas possible d'y entrer des lettres		
	La fonctionnalité de modification est accessible uniquement par un administrateur.	Un utilisateur simple n'a pas le bouton de modification affiché sur son écran.		

Fonctionnalité	Procédure	Résultat attendu	Résultat	Commentaire
Ajout d'écurie	Ajouter une écurie avec nom de type texte.	L'écurie est créée et se retrouve dans la liste des écuries.		
	Ajouter une écurie avec un nom contenant des chiffres.	Un message disant que le nom n'est pas correct et l'écurie ne se crée pas.		L'écurie est quand même créée
	La fonctionnalité d'ajout est accessible uniquement par un administrateur.	Un utilisateur simple n'a pas le bouton d'ajout affiché sur son écran.		
Modification d'écurie	Modifier une écurie avec un nom de type texte.	Seul l'écurie visée par la modification est modifiée.		
	Les données avant modifications de l'écurie se retrouvent dans les bons champs.			
	Modifier une écurie avec un nom contenant des chiffres.	Un message disant que le nom n'est pas correct et l'écurie n'est pas modifiée		L'écurie est quand même modifiée
	La fonctionnalité de modification est accessible uniquement par un administrateur.	Un utilisateur simple n'a pas le bouton de modification affiché sur son écran.		

Fonctionnalité	Procédure	Résultat attendu	Résultat	Commentaire
Ajout de circuit	Ajouter un circuit avec un nom de type texte, un ordre et une longueur du circuit de type nombre.	Le circuit est créé et se retrouve dans la liste des circuits. L'ordre du circuit entré est respecté et les autres circuits sont modifiés		La nationalité doit être choisie dans la liste déroulante
	Entre des lettres dans la longueur ou l'ordre du circuit	Il n'est pas possible d'y entrer des lettres		
	La fonctionnalité d'ajout est accessible uniquement par un administrateur.	Un utilisateur simple n'a pas le bouton d'ajout affiché sur son écran.		
Modification de circuit	Modifier un circuit avec un nom de type texte, un ordre et une longueur du circuit de type nombre.	Seul le circuit visé par la modification est modifié.		
	Les données avant modifications du circuit se retrouvent dans les bons champs.			
	Entre des lettres dans la longueur ou l'ordre du circuit	Il n'est pas possible d'y entrer des lettres		
	La fonctionnalité de modification est accessible uniquement par un administrateur.	Un utilisateur simple n'a pas le bouton de modification affiché sur son écran.		

Fonctionnalité	Procédure	Résultat attendu	Résultat	Commentaire
Début de saison	Ne pas choisir d'écurie lors de la sélection	Un message d'erreur disant qu'il faut sélectionner 1 écurie min est affiché		

	Sélectionner 0 ou 1 pilote	Un message d'erreur disant qu'il faut sélectionner 2 pilotes exactement est affiché		
Résultat de course	Simuler une course et passer du résultat aux classements	Les points sont bien additionnés, les pilotes et écuries sont dans l'ordre décroissant de leur points		
Prochaine course	Passer de la course 1 à la course 2	La « prochaine course » affichée est bien celle qui a lieu après la course 1		
Sauvegarde	Commencer une saison, faire la première course, fermer l'onglet à la page des classement, réaccéder au site et sélectionner « Reprendre la saison en cours »	On se retrouve sur le page des classements de la première course, les pilotes et l'écurie choisie sont les mêmes qu'avant et les points sont pareils		

3.4 Erreurs restantes

Les erreurs restantes sont majoritairement sur la vérification du type de données entrées dans les formulaires. Par exemple le fait qu'un administrateur peut entrer des lettre dans le nom d'un pilote. La fonctionnalité reste utilisable sans retourner de message d'erreur mais n'est pas idéal. Il faudrait ajouter une vérification pour chaque champs lorsque le formulaire est soumis et y afficher un message d'erreur lorsque nécessaire.

3.5 Liste des documents fournis

Lister les documents fournis au client avec votre produit, en indiquant les numéros de versions

- *le rapport de projet*
- *le manuel d'Installation (en annexe)*
- *le manuel d'Utilisation avec des exemples graphiques (en annexe)*
- *autres...*

4 Conclusions

4.1 Objectifs

Les objectifs fixés par le cahiers des charges ont été atteint. Le produit final est utilisable, déployé et a un visuel plutôt agréable. La planification et l'analyse faite en début de projet a grandement aidé à fixer les objectifs à réaliser durant le projet, l'ordre à laquelle les réaliser.

4.2 Bilan personnel

Ce projet m'a permis de mettre en pratique ce que j'ai appris durant mes années de formation d'informaticien et surtout de me prouver que je peux développer des petites application web avec un backend et frontend en utilisant react et expressJS en ayant appris en autodidacte. Ce côté « découverte » a rendu le projet un peu plus excitant et passionnant et je n'aurais pas ressenti tout ça si j'avais effectué ce projet en PHP. Le déploiement de l'application sur le serveur de l'ETML a aussi été une étape intéressante. Cela donne un sens à ce que l'on a développé et est une étape que l'on n'a jamais vu durant la formation malgré son importance. C'est toujours un peu plus

cool d'avoir son application disponible sur n'importe quel poste et à tout moment. Ça change d'avoir son application uniquement en local.

4.3 Difficultés rencontrées

Les difficultés rencontrées étaient majoritairement sur le formatage des données avant de les utiliser. La communication entre les différents composant et les variables étaient aussi quelque chose d'assez complexe.

Une des grandes difficultés rencontrées durant le projet et qui m'a pris 1 journée de travail à résoudre était un problème lors du déploiement de l'application.

Le problème initial était que l'application n'arrivait pas à différencier les routes backend et les routes frontend. Cette erreur faisait que si j'étais sur la page */mainPage*, elle était bien affichée mais en cas de rafraîchissement de la page, elle envoyait une requête au backend sur la route */mainPage* qui n'existe pas. Le problème était que les requêtes ne pointaient pas sur le fichier index du build (présent dans le dossier *.dist*). La résolution de ce problème en a mené à un autre. Lorsque je démarrais mon application et que je me rendais sur le site web, il y avait une erreur 502. Au début je pensais que cela venait de ce que j'avais modifié suite au problème précédent. J'étais donc bloqué entre 2 problèmes sans pouvoir résoudre les 2 en même temps. De plus, rien ne semblait être différent de l'environnement de mon pré-TPI. Je n'avais donc aucune idée d'où cela venait. J'ai finalement découvert un fichier logs qui affichait une erreur rencontrée avec la dépendance *regexp* (qui sert à *react-dom-router*). Après plusieurs heures de recherche et tentative, je suis tombé sur un forum disant que la dernière version d'*expressJS* apportait quelques problème. J'ai donc essayé avec une version plus ancienne ce qui a résolu le problème.

4.4 Améliorations et évolutions possibles

Plusieurs améliorations ou changement peuvent être envisagés. Tout d'abord la vérification des champs dans les formulaires serait une des premières choses à faire.

Dans le local storage du navigateur, les données de course y sont stockées.

https://aki.w3.pm2etml.ch	
Origin	https://aki.w3.pm2etml.ch
Key	Value
gameSettings	[{"id":34,"firstname":"Max","last
gameState	endSeason
inGame	true
raceResult	[{"id":30,"firstname":"Oscar","la
userToken	eyJhbGciOiJIUzI1NiIsInR5cCI6Ikp

Il serait important d'utiliser le JWT sur ces données comme pour le *userToken* afin de s'assurer que le joueur ne modifie pas lui-même ces données.

Pour l'instant, les seules pays disponible lors de la création d'un pilote ou d'un circuit ne sont pas complets. Il faudrait simplement ajouter une liste de pays plus complète ou au moins ajouter une *pays par défaut*.

Ensuite plusieurs ajouts peuvent être implémenter pour augmenter le côté jouabilité de l'application comme ajouter un budget maximum pour le joueur et ajouter un prix au pilotes. Le joueur devra donc faire un choix stratégique entre le prix et le niveau du pilote. Ensuite une amélioration de la mécanique de *rating* du pilote. S'inspirer du système des jeux F1 de EA sports avec des attributs comme les reflex, la capacité à dépasser, etc... S'ajouterait à ça une amélioration de la simulation de course prenant en compte les nouveaux attributs des pilotes. Cela rendrait les simulations plus réels.

Pour l'instant, lors de la simulation de la course, il n'y a qu'un simple écran de chargement. Il serait intéressant d'y ajouter une animation des tours effectués et lors d'accidents, de les afficher sur le moment et d'y montrer un drapeau jaune.

5 Annexes

5.1 Résumé du rapport du TPI / version succincte de la documentation

5.2 Sources – Bibliographie

Liste des livres utilisés (Titre, auteur, date), des sites Internet (URL) consultés, des articles (Revue, date, titre, auteur)... Et de toutes les aides externes (noms)

5.3 Journal de travail

Date	Durée	Activité	Remarques

5.4 Manuel d'Installation

5.5 Manuel d'Utilisation

5.6 Archives du projet

Media, ... dans une fourre en plastique

6 Table des illustrations

Figure 1: Planification initiale	8
Figure 2: Zoning page principale	16
Figure 3: Zoning liste pilotes.....	17
Figure 4: Zoning liste écuries.....	18
Figure 5: Maquette page principale	19
Figure 6: Maquette liste pilotes.....	20
Figure 7: Maquette liste écuries.....	21
Figure 8: Maquette classement après-course.....	22
Figure 9: Logigramme authentification	32
Figure 10: Exemple d'une route.....	38
Figure 11: Composition composants liste pilotes.....	39
Figure 12: Interface d'un objet pilote.....	40
Figure 13: Affichage de la liste des pilotes	41

7 Glossaire